Model6

May 2, 2026

1 Model 6 - PPO Optimized

1.0.1 Cell 1: Install (if needed), restart kernel, then Cell 2

[1]:
```
# Cell 1: Install
import subprocess, sys
subprocess.check_call([sys.executable, '-m', 'pip', 'install',
    'torch', '--index-url', 'https://download.pytorch.org/whl/cu128', '-q'])
subprocess.check_call([sys.executable, '-m', 'pip', 'install',
    'numpy', 'transformers', 'datasets', 'sentence-transformers', 'accelerate',␣
 ↪'-q'])
import torch
print('torch:', torch.__version__)
print('cuda:', torch.cuda.is_available())
if torch.cuda.is_available(): print('gpu:', torch.cuda.get_device_name(0))
print('DONE - restart kernel then run Cell 2')
```

```
torch: 2.11.0+cu128
cuda: True
gpu: NVIDIA H200 NVL
DONE - restart kernel then run Cell 2
```

[2]:
```
# ============================================================
# Model 6: PPO Optimized + Batched GPU Parallelism

# ============================================================

import os
import re
import json
import copy
import time
import random
import shutil
import hashlib
from dataclasses import dataclass
from typing import Dict, List, Tuple, Optional
```

```python
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
from sentence_transformers import SentenceTransformer


# ============================================================
# Config
# ============================================================

@dataclass
class Config:
    # Dataset
    train_subset: int = 300
    test_subset: int = 100

    # Models
    llm_name: str = "Qwen/Qwen2.5-1.5B-Instruct"
    embed_name: str = "sentence-transformers/all-MiniLM-L6-v2"

    # PARALLELISM: how many questions to process at once on GPU
    # H200 has 94GB VRAM so 16 is safe, increase to 32 if no OOM
    llm_batch_size: int = 16

    # Oracle budgets: only 128-220 where model actually works
    # Removed 96 and 112 because model fails 70-74% there
    oracle_budgets: Tuple[int, ...] = (
        128, 144, 160, 168, 176, 184, 192, 200, 210, 220
    )

    # Supervised warm-start
    warmstart_epochs: int = 15
    warmstart_learning_rate: float = 5e-4

    # PPO specific
    rl_epochs: int = 10
    ppo_epochs: int = 2          # FIX: reduced from 4 to 2 - prevents␣
↪over-updating per rollout
    ppo_epsilon: float = 0.2     # clip range: ratio stays in [0.8, 1.2]
    ppo_batch_size: int = 32     # mini-batch size for PPO updates
    policy_learning_rate: float = 3e-4
    value_learning_rate: float = 8e-4
    entropy_coef: float = 0.01   # reduced from 0.02, ClipFrac was too erratic
```

```
    value_loss_coef: float = 0.5
    grad_clip_norm: float = 1.0

    # Reward: rebalanced penalty so 176-token correct gets positive reward
    # At 176 tokens: 1.0 + 0.25 - 0.0005*176 = 1.162  (correct)
    # At 128 tokens: 1.0 + 0.25 - 0.0005*128 = 1.186  (correct)
    # Small gap encourages efficiency without punishing longer budgets
    reward_token_penalty: float = 0.0005
    exact_match_bonus: float = 0.25

    # Budget range: min raised to 128 (below this model fails reliably)
    min_budget: int = 128
    max_budget: int = 220

    # Answer generation
    answer_max_new_tokens: int = 20
    do_sample: bool = False
    temperature: float = 0.0

    # RL warmup exploration in 160-220 where model succeeds
    warmup_epochs_rl: int = 2
    warmup_mix_rl: float = 0.25
    warmup_min_budget_high: int = 160

    # Oracle blend in PPO: supervised signal to keep deterministic head moving
    use_oracle_blend: bool = True
    oracle_blend_weight: float = 0.4   # increased: keeps deterministic head learning

    # Hardware
    seed: int = 42
    device: str = "cuda" if torch.cuda.is_available() else "cpu"

    # Logging
    print_every: int = 20
    debug_examples: int = 3
    eval_print_every: int = 10

    # Cache
    cache_dir: str = "./cache_ppo_v13"
    clear_cache_on_start: bool = True
    oracle_cache_path: str = "oracle_labels_v13.json"

    # Saved models
    best_warmstart_path: str = "best_warmstart_v13.pt"
    best_warmstart_acc_path: str = "best_warmstart_acc_v13.pt"
    best_ppo_path: str = "best_ppo_v13.pt"
```

```
    best_ppo_acc_path: str = "best_ppo_acc_v13.pt"
    best_value_path: str = "best_value_v13.pt"
    final_policy_path: str = "final_policy_v13.pt"
    final_value_path: str = "final_value_v13.pt"

    # Baselines
    baseline_budgets: Tuple[int, ...] = (128, 144, 160, 168, 176, 184, 192,␣
 ↪210, 220)


CFG = Config()


# ============================================================
# Utilities
# ============================================================

def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)


def ensure_dir(path):
    os.makedirs(path, exist_ok=True)


def reset_cache_dir(cache_dir, retries=5, delay=0.5):
    if not os.path.exists(cache_dir):
        os.makedirs(cache_dir, exist_ok=True)
        return
    last_error = None
    for _ in range(retries):
        try:
            if os.path.exists(cache_dir):
                shutil.rmtree(cache_dir, ignore_errors=False)
            if os.path.exists(cache_dir):
                for root, dirs, files in os.walk(cache_dir, topdown=False):
                    for f in files:
                        try: os.remove(os.path.join(root, f))
                        except FileNotFoundError: pass
                    for d in dirs:
                        try: os.rmdir(os.path.join(root, d))
                        except OSError: pass
                if os.path.exists(cache_dir): os.rmdir(cache_dir)
```

```
            break
        except OSError as e:
            last_error = e
            time.sleep(delay)
    if os.path.exists(cache_dir):
        raise RuntimeError(f"Failed to clear cache: {cache_dir}") from␣
↪last_error
    os.makedirs(cache_dir, exist_ok=True)


def clamp_budget(x, min_budget, max_budget):
    return int(max(min_budget, min(max_budget, round(x))))


def denormalize_budget(x, min_budget, max_budget):
    return x * (max_budget - min_budget) + min_budget


def budget_to_normalized(budget, min_budget, max_budget):
    return float(budget - min_budget) / float(max_budget - min_budget)


def clean_model_output(text):
    if text is None: return ""
    cleaned = text
    for marker in ["\nHuman:", "\nAssistant:", "\nUser:", "\nSystem:",
                   "Human:", "Assistant:", "User:", "System:",
                   "You are an AI assistant", "<|im_end|>", "<|endoftext|>"]:
        if marker in cleaned:
            cleaned = cleaned.split(marker)[0]
    return cleaned.strip()


# ============================================================
# GSM8K helpers
# ============================================================

def extract_gsm8k_final_answer(answer_text):
    if "####" in answer_text:
        return answer_text.split("####")[-1].strip()
    return answer_text.strip()


def normalize_numeric_string(text):
    if text is None: return ""
    text = text.strip().replace(",", "").replace("$", "").replace("%", "").
↪strip()
```

```python
    matches = re.findall(r"-?\d+(?:\.\d+)?", text)
    if not matches: return text.lower().strip()
    return matches[-1]


def parse_final_answer(output_text):
    text = clean_model_output(output_text)
    for pattern in [
        r"FINAL ANSWER\s*:\s*(-?\d+(?:\.\d+)?)",
        r"Final Answer\s*:\s*(-?\d+(?:\.\d+)?)",
        r"Answer\s*:\s*(-?\d+(?:\.\d+)?)",
    ]:
        m = re.search(pattern, text, re.IGNORECASE)
        if m: return normalize_numeric_string(m.group(1))
    lines = [l.strip() for l in text.splitlines() if l.strip()]
    first_line = lines[0] if lines else text.strip()
    m = re.search(r"-?\d+(?:\.\d+)?", first_line)
    if m: return normalize_numeric_string(m.group(0))
    matches = re.findall(r"-?\d+(?:\.\d+)?", text)
    if matches: return normalize_numeric_string(matches[0])
    return normalize_numeric_string(text)


def is_correct(pred, gold):
    return int(pred == gold)


def compute_reward(pred, gold, thinking_tokens, cfg):
    exact = float(is_correct(pred, gold))
    reward = exact - cfg.reward_token_penalty * float(thinking_tokens)
    if exact > 0.5:
        reward += cfg.exact_match_bonus
    return reward, exact


# ============================================================
# Prompts
# ============================================================

def build_thinking_prompt(question):
    return (
        "Solve the following grade-school math problem carefully.\n"
        "Reason step by step. Keep reasoning concise and relevant.\n"
        "Do not start a conversation. Only produce reasoning.\n\n"
        f"Question: {question}\n\nReasoning:"
    )
```

```python
def build_answer_prompt(question, thinking_text):
    return (
        "Use the reasoning below to produce the final numeric answer.\n"
        "Return only: FINAL ANSWER: <number>\n\n"
        f"Question: {question}\n\nReasoning:\n{thinking_text}\n\nFINAL ANSWER:"
    )


# ============================================================
# Caching
# ============================================================

def make_cache_key(stage, question, extra, model_name):
    raw = f"{stage}|||{model_name}|||{question}|||{extra}"
    return hashlib.md5(raw.encode()).hexdigest()


def load_cache(cache_dir, key):
    path = os.path.join(cache_dir, f"{key}.json")
    if os.path.exists(path):
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    return None


def save_cache(cache_dir, key, value):
    path = os.path.join(cache_dir, f"{key}.json")
    with open(path, "w", encoding="utf-8") as f:
        json.dump(value, f, ensure_ascii=False, indent=2)


# ============================================================
# LLM Reasoner with BATCHED GPU inference
# This is the main speedup vs Model 5
# Instead of 1 question at a time, processes batch_size questions
# simultaneously on the GPU - uses the H200 properly
# ============================================================

class LLMReasoner:
    def __init__(self, model_name, device, batch_size=16):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        if self.tokenizer.pad_token is None:
            self.tokenizer.pad_token = self.tokenizer.eos_token

        # LEFT padding is required for decoder-only models in batched mode
        # Without this, the model attends to wrong positions
```

```
        self.tokenizer.padding_side = "left"

        self.model = AutoModelForCausalLM.from_pretrained(
            model_name,
            torch_dtype=torch.float16,
            device_map="auto"
        )
        self.model.eval()
        self.device = device
        self.batch_size = batch_size

        if hasattr(self.model, "generation_config") and self.model.generation_config:
            self.model.generation_config.do_sample = False
            self.model.generation_config.temperature = None
            self.model.generation_config.top_p = None
            self.model.generation_config.top_k = None

    def generate(self, prompt, max_new_tokens, do_sample=False, temperature=0.0):
        """Single generation - keeps backward compatibility with cache system."""
        results = self.generate_batch([prompt], max_new_tokens, do_sample, temperature)
        return results[0]

    def generate_batch(self, prompts, max_new_tokens, do_sample=False, temperature=0.0):
        """
        BATCHED GPU INFERENCE - the key parallelism speedup.

        How it works:
          - Takes a list of prompts
          - Tokenizes them all together with padding
          - GPU processes ALL of them in one forward pass
          - Returns results for all prompts

        Why this is faster:
          - Sequential: GPU runs at 5% utilization (mostly idle waiting)
          - Batched: GPU runs at 70-90% utilization
          - Speedup: 4-8x depending on batch size
        """
        all_results = []

        for i in range(0, len(prompts), self.batch_size):
            batch = prompts[i : i + self.batch_size]
```

```
            # Tokenize with padding so all sequences are same length
            inputs = self.tokenizer(
                batch,
                return_tensors="pt",
                padding=True,
                truncation=True,
                max_length=600,
            ).to(self.device)

            input_len = inputs["input_ids"].shape[1]

            gen_kwargs = {
                "max_new_tokens": max_new_tokens,
                "do_sample": do_sample,
                "pad_token_id": self.tokenizer.eos_token_id,
                "eos_token_id": self.tokenizer.eos_token_id,
                "attention_mask": inputs["attention_mask"],
            }
            if do_sample:
                gen_kwargs["temperature"] = temperature

            with torch.no_grad():
                # GPU processes entire batch at once
                outputs = self.model.generate(inputs["input_ids"], **gen_kwargs)

            for output in outputs:
                gen_ids = output[input_len:]
                text = clean_model_output(
                    self.tokenizer.decode(gen_ids, skip_special_tokens=True)
                )
                all_results.append({"text": text, "tokens_used": len(gen_ids)})

        return all_results


# ============================================================
# Two-stage example run (single, uses cache)
# ============================================================

def run_two_stage_example(question, gold_answer, thinking_budget, llm, cfg):
    ensure_dir(cfg.cache_dir)

    # Stage 1: Thinking
    think_key = make_cache_key("think", question, f"budget={thinking_budget}",
 ↪cfg.llm_name)
    cached = load_cache(cfg.cache_dir, think_key)
    if cached:
```

```python
        thinking_text = clean_model_output(cached["text"])
        thinking_tokens = cached["tokens_used"]
    else:
        r = llm.generate(build_thinking_prompt(question), thinking_budget)
        thinking_text = clean_model_output(r["text"])
        thinking_tokens = r["tokens_used"]
        save_cache(cfg.cache_dir, think_key, {"text": thinking_text,␣
↪"tokens_used": thinking_tokens})

    # Stage 2: Answer
    answer_key = make_cache_key("answer", question, thinking_text, cfg.llm_name)
    cached = load_cache(cfg.cache_dir, answer_key)
    if cached:
        answer_text = clean_model_output(cached["text"])
        answer_tokens = cached["tokens_used"]
    else:
        r = llm.generate(build_answer_prompt(question, thinking_text), cfg.
↪answer_max_new_tokens)
        answer_text = clean_model_output(r["text"])
        answer_tokens = r["tokens_used"]
        save_cache(cfg.cache_dir, answer_key, {"text": answer_text,␣
↪"tokens_used": answer_tokens})

    pred = parse_final_answer(answer_text)
    reward, exact = compute_reward(pred, gold_answer, thinking_tokens, cfg)
    return {
        "prediction": pred, "gold": gold_answer, "correct": int(exact),
        "thinking_tokens_used": thinking_tokens, "answer_tokens_used":␣
↪answer_tokens,
        "reward": reward, "thinking_text": thinking_text, "answer_text":␣
↪answer_text,
        "thinking_budget": thinking_budget,
    }


# ============================================================
# BATCHED two-stage run for Oracle and RL rollouts
# Processes multiple questions at once using GPU parallelism
# ============================================================

def run_batch_two_stage(questions, gold_answers, thinking_budget, llm, cfg):
    """
    Run two-stage inference for a BATCH of questions at the same budget.

    Step 1: Check cache for all questions
    Step 2: Run LLM on uncached questions in ONE batched GPU call
    Step 3: Run answer extraction in ONE batched GPU call
```

```
    Step 4: Save new results to cache

    This replaces the sequential for-loop that caused the 4-hour runtime.
    """
    ensure_dir(cfg.cache_dir)
    n = len(questions)

    # --- Stage 1: Thinking ---
    thinking_texts = [None] * n
    thinking_tokens_list = [None] * n
    uncached_think_idx = []
    uncached_think_prompts = []

    # Check cache first for all questions
    for i, q in enumerate(questions):
        key = make_cache_key("think", q, f"budget={thinking_budget}", cfg.
↪llm_name)
        cached = load_cache(cfg.cache_dir, key)
        if cached:
            thinking_texts[i] = clean_model_output(cached["text"])
            thinking_tokens_list[i] = cached["tokens_used"]
        else:
            uncached_think_idx.append(i)
            uncached_think_prompts.append(build_thinking_prompt(q))

    # Batch GPU call for all uncached thinking
    if uncached_think_prompts:
        batch_results = llm.generate_batch(uncached_think_prompts,␣
↪thinking_budget)
        for idx, result in zip(uncached_think_idx, batch_results):
            thinking_texts[idx] = clean_model_output(result["text"])
            thinking_tokens_list[idx] = result["tokens_used"]
            key = make_cache_key("think", questions[idx],␣
↪f"budget={thinking_budget}", cfg.llm_name)
            save_cache(cfg.cache_dir, key, {
                "text": thinking_texts[idx],
                "tokens_used": thinking_tokens_list[idx]
            })

    # --- Stage 2: Answer extraction ---
    answer_texts = [None] * n
    answer_tokens_list = [None] * n
    uncached_ans_idx = []
    uncached_ans_prompts = []

    for i in range(n):
```

```python
        key = make_cache_key("answer", questions[i], thinking_texts[i], cfg.
↪llm_name)
        cached = load_cache(cfg.cache_dir, key)
        if cached:
            answer_texts[i] = clean_model_output(cached["text"])
            answer_tokens_list[i] = cached["tokens_used"]
        else:
            uncached_ans_idx.append(i)
            uncached_ans_prompts.append(
                build_answer_prompt(questions[i], thinking_texts[i])
            )

    # Batch GPU call for all uncached answers
    if uncached_ans_prompts:
        batch_results = llm.generate_batch(uncached_ans_prompts, cfg.
↪answer_max_new_tokens)
        for idx, result in zip(uncached_ans_idx, batch_results):
            answer_texts[idx] = clean_model_output(result["text"])
            answer_tokens_list[idx] = result["tokens_used"]
            key = make_cache_key("answer", questions[idx], thinking_texts[idx],␣
↪cfg.llm_name)
            save_cache(cfg.cache_dir, key, {
                "text": answer_texts[idx],
                "tokens_used": answer_tokens_list[idx]
            })

    # --- Compute rewards ---
    results = []
    for i in range(n):
        pred = parse_final_answer(answer_texts[i])
        reward, exact = compute_reward(pred, gold_answers[i],␣
↪thinking_tokens_list[i], cfg)
        results.append({
            "prediction": pred, "gold": gold_answers[i], "correct": int(exact),
            "thinking_tokens_used": thinking_tokens_list[i],
            "answer_tokens_used": answer_tokens_list[i],
            "reward": reward, "thinking_text": thinking_texts[i],
            "answer_text": answer_texts[i], "thinking_budget": thinking_budget,
        })
    return results


# ============================================================
# Featurizer
# ============================================================

class QuestionFeaturizer:
```

```python
    def __init__(self, embed_model_name, device):
        self.embedder = SentenceTransformer(embed_model_name, device=device)

    def handcrafted_features(self, question):
        q = question.lower()
        tokens = q.split()
        keywords = [
            "total", "left", "remaining", "each", "every", "more", "less",
            "after", "before", "twice", "half", "times", "altogether",
            "difference", "sum", "product", "cost", "price", "sold",
            "per", "average", "equal", "share", "then", "gave", "bought",
            "earned", "minutes", "hours", "days", "weeks", "fraction",
            "ratio", "percent", "another", "still", "now", "together"
        ]
        feats = np.array(
            [
                len(tokens),
                len(re.findall(r"\d+", q)),
                len(q),
                max(1, len(re.findall(r"[.!?]", q))),
                1.0 if any(w in q for w in ["more", "sum", "altogether", "total", "together"]) else 0.0,
                1.0 if any(w in q for w in ["left", "remaining", "difference", "less", "still"]) else 0.0,
                1.0 if any(w in q for w in ["each", "every", "times", "product", "twice"]) else 0.0,
                1.0 if any(w in q for w in ["per", "average", "equally", "half", "share", "ratio"]) else 0.0,
                1.0 if any(w in q for w in ["then", "after", "before", "altogether", "remaining", "now"]) else 0.0,
                1.0 if any(w in q for w in ["minutes", "hours", "days", "weeks"]) else 0.0,
                1.0 if "$" in q or any(w in q for w in ["dollar", "dollars", "cost", "price"]) else 0.0,
                1.0 if any(w in q for w in ["half", "fraction", "percent", "ratio"]) else 0.0,
            ] + [1.0 if kw in q else 0.0 for kw in keywords],
            dtype=np.float32
        )
        return feats

    def encode(self, question):
        emb = self.embedder.encode(
            question, convert_to_numpy=True, normalize_embeddings=True
        ).astype(np.float32)
```

```python
        return np.concatenate([emb, self.handcrafted_features(question)],␣
↪axis=0)

    def encode_batch(self, questions):
        """Encode multiple questions at once for featurizer speedup."""
        embs = self.embedder.encode(
            questions, convert_to_numpy=True,
            normalize_embeddings=True, batch_size=64
        ).astype(np.float32)
        hand = np.stack([self.handcrafted_features(q) for q in questions])
        return np.concatenate([embs, hand], axis=1)


# ============================================================
# Policy + Value Networks
# ============================================================

class ContinuousBudgetPolicy(nn.Module):
    def __init__(self, input_dim, hidden_dim=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
        )
        self.mean_head = nn.Linear(hidden_dim, 1)
        # log_std = 0.5 gives std ~1.65 for initial exploration
        self.log_std = nn.Parameter(torch.tensor([0.5], dtype=torch.float32))

    def forward(self, x):
        h = self.net(x)
        mean = self.mean_head(h)
        return mean, self.log_std.expand_as(mean)

    def sample_budget(self, x):
        mean, log_std = self.forward(x)
        dist = torch.distributions.Normal(mean, torch.exp(log_std))
        raw = dist.rsample()
        normalized = torch.sigmoid(raw)
        log_prob = dist.log_prob(raw).sum(dim=-1)
        entropy = dist.entropy().sum(dim=-1)
        return normalized, log_prob, entropy

    def get_log_prob(self, x, normalized_budgets):
        """
        Get log probability for given normalized budgets.
        Used in PPO to compute ratio between new and old policy.
        """
```

```
        mean, log_std = self.forward(x)
        dist = torch.distributions.Normal(mean, torch.exp(log_std))
        # Inverse sigmoid to get raw values
        raw = torch.logit(normalized_budgets.unsqueeze(-1).clamp(1e-6, 1 -␣
↪1e-6))
        log_prob = dist.log_prob(raw).sum(dim=-1)
        entropy = dist.entropy().sum(dim=-1)
        return log_prob, entropy

    def deterministic_budget(self, x):
        mean, _ = self.forward(x)
        return torch.sigmoid(mean)


class ValueNetwork(nn.Module):
    def __init__(self, input_dim, hidden_dim=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, 1),
        )

    def forward(self, x):
        return self.net(x).squeeze(-1)


# ============================================================
# Evaluation
# ============================================================

def evaluate_fixed_budget(data, llm, cfg, fixed_budget):
    corrects, tokens, rewards = [], [], []
    questions = [item["question"] for item in data]
    gold_answers = [
        normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        for item in data
    ]
    print(f"\nEvaluating fixed budget = {fixed_budget}...")

    # Process in batches
    for i in range(0, len(questions), cfg.llm_batch_size):
        batch_q = questions[i : i + cfg.llm_batch_size]
        batch_g = gold_answers[i : i + cfg.llm_batch_size]
        results = run_batch_two_stage(batch_q, batch_g, fixed_budget, llm, cfg)
        for r in results:
            corrects.append(r["correct"])
```

```python
            tokens.append(r["thinking_tokens_used"])
            rewards.append(r["reward"])
        done = min(i + cfg.llm_batch_size, len(questions))
        if done % cfg.eval_print_every == 0 or done == len(questions):
            print(f"  Fixed {fixed_budget} | {done}/{len(questions)} | "
                  f"Acc={np.mean(corrects):.4f} | AvgReward={np.mean(rewards):.
↪4f}")

    return {
        "accuracy": float(np.mean(corrects)),
        "avg_thinking_tokens": float(np.mean(tokens)),
        "avg_reward": float(np.mean(rewards)),
        "budget": fixed_budget
    }


def evaluate_policy_deterministic(data, featurizer, policy, llm, cfg):
    policy.eval()
    corrects, tokens, rewards, budgets = [], [], [], []

    questions = [item["question"] for item in data]
    gold_answers = [
        normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        for item in data
    ]

    # Encode all features at once
    all_feats = featurizer.encode_batch(questions)

    with torch.no_grad():
        feats_tensor = torch.tensor(all_feats, dtype=torch.float32, device=cfg.
↪device)
        norm_budgets = policy.deterministic_budget(feats_tensor).squeeze(-1)
        budget_list = [
            clamp_budget(
                denormalize_budget(nb.item(), cfg.min_budget, cfg.max_budget),
                cfg.min_budget, cfg.max_budget
            )
            for nb in norm_budgets
        ]

    # Group questions by budget for batched inference
    from collections import defaultdict
    budget_groups = defaultdict(list)
    for i, b in enumerate(budget_list):
        budget_groups[b].append(i)
```

```python
    results_by_idx = {}
    for budget, indices in budget_groups.items():
        batch_q = [questions[i] for i in indices]
        batch_g = [gold_answers[i] for i in indices]
        batch_results = run_batch_two_stage(batch_q, batch_g, budget, llm, cfg)
        for i, r in zip(indices, batch_results):
            results_by_idx[i] = r

    for i in range(len(data)):
        r = results_by_idx[i]
        corrects.append(r["correct"])
        tokens.append(r["thinking_tokens_used"])
        rewards.append(r["reward"])
        budgets.append(budget_list[i])

    if corrects:
        print(f"  Det eval complete | Acc={np.mean(corrects):.4f} | "
              f"AvgBudget={np.mean(budgets):.1f} | StdBudget={np.std(budgets):.
↪1f} | "
              f"Range=[{np.min(budgets):.0f}, {np.max(budgets):.0f}]")

    return {
        "accuracy": float(np.mean(corrects)),
        "avg_thinking_tokens": float(np.mean(tokens)),
        "avg_reward": float(np.mean(rewards)),
        "avg_budget": float(np.mean(budgets)),
        "min_budget": float(np.min(budgets)),
        "max_budget": float(np.max(budgets)),
        "std_budget": float(np.std(budgets)),
    }


# ============================================================
# Oracle builder using BATCHED inference
# Model 5 took ~2.5 hours for this step alone
# With batching: processes 16 questions per budget in one GPU call
# Expected speedup: 3-5x (from ~2.5 hours to ~30-45 min)
# ============================================================

def build_oracle_budget_labels(train_data, llm, cfg):
    if os.path.exists(cfg.oracle_cache_path):
        print(f"\nLoading oracle labels from {cfg.oracle_cache_path}...")
        with open(cfg.oracle_cache_path, "r", encoding="utf-8") as f:
            return json.load(f)

    print("\nBuilding oracle labels with BATCHED GPU inference...")
    questions = [item["question"] for item in train_data]
```

```
    gold_answers = [
        normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        for item in train_data
    ]
    n = len(questions)

    # Initialize records
    oracle_records = [
        {"question": q, "gold_answer": g, "budget_rewards": {}, "any_correct":␣
↪False}
        for q, g in zip(questions, gold_answers)
    ]

    # For each budget, process ALL 300 questions in batches
    # This is much faster than the nested loop in Model 5
    for budget in cfg.oracle_budgets:
        print(f"  Trying budget={budget} for all {n} questions...")
        t0 = time.time()

        for i in range(0, n, cfg.llm_batch_size):
            batch_q = questions[i : i + cfg.llm_batch_size]
            batch_g = gold_answers[i : i + cfg.llm_batch_size]
            batch_results = run_batch_two_stage(batch_q, batch_g, budget, llm,␣
↪cfg)

            for j, result in enumerate(batch_results):
                idx = i + j
                oracle_records[idx]["budget_rewards"][str(budget)] = {
                    "reward": float(result["reward"]),
                    "correct": int(result["correct"]),
                    "thinking_tokens_used": int(result["thinking_tokens_used"]),
                    "prediction": result["prediction"],
                }
                if result["correct"] == 1:
                    oracle_records[idx]["any_correct"] = True

        elapsed = time.time() - t0
        correct_at_budget = sum(
            1 for r in oracle_records
            if r["budget_rewards"].get(str(budget), {}).get("correct", 0) == 1
        )
        print(f"    Budget {budget} done in {elapsed:.1f}s | "
              f"Correct: {correct_at_budget}/{n} ({100*correct_at_budget/n:.
↪1f}%)")

    # Assign best budget per question
    # FIX from Model 5: use HIGHEST REWARD budget, not minimum correct
```

```
    # Model 5 assigned 57% of questions to budget=96 which skewed everything
    for record in oracle_records:
        br = record["budget_rewards"]
        best_budget = max(br.keys(), key=lambda b: br[b]["reward"])
        record["best_budget"] = int(best_budget)
        record["best_reward"] = float(br[best_budget]["reward"])

    # Print distribution to verify it looks reasonable
    best_budgets = [r["best_budget"] for r in oracle_records]
    solvable = [r["any_correct"] for r in oracle_records]
    print(f"\nOracle stats: mean={np.mean(best_budgets):.1f}, "
          f"std={np.std(best_budgets):.1f}, "
          f"solvable={np.mean(solvable):.3f}")
    for b in sorted(set(best_budgets)):
        count = sum(1 for x in best_budgets if x == b)
        print(f"  budget={b}: {count} questions ({100*count/len(best_budgets):.
↪1f}%)")

    with open(cfg.oracle_cache_path, "w", encoding="utf-8") as f:
        json.dump(oracle_records, f, ensure_ascii=False, indent=2)
    print(f"Saved oracle labels to {cfg.oracle_cache_path}")
    return oracle_records


# ============================================================
# Supervised warm-start
# ============================================================

def warmstart_policy(oracle_records, test_data, featurizer, policy, llm, cfg):
    print("\nStarting supervised warm-start...")
    optimizer = optim.Adam(policy.parameters(), lr=cfg.warmstart_learning_rate)
    mse_loss = nn.MSELoss(reduction="none")

    all_budgets = np.array([r["best_budget"] for r in oracle_records], dtype=np.
↪float32)
    budget_mean = float(all_budgets.mean())
    budget_std = float(all_budgets.std()) + 1e-8

    best_reward = -1e9
    best_accuracy = -1.0

    for epoch in range(cfg.warmstart_epochs):
        policy.train()
        random.shuffle(oracle_records)
        losses, pred_budgets, target_budgets = [], [], []

        for step_idx, record in enumerate(oracle_records):
```

```
            target_budget = int(record["best_budget"])
            x = torch.tensor(
                featurizer.encode(record["question"]),
                dtype=torch.float32, device=cfg.device
            ).unsqueeze(0)

            pred_norm = policy.deterministic_budget(x)
            target_norm = torch.tensor(
                [[budget_to_normalized(target_budget, cfg.min_budget, cfg.
↪max_budget)]],
                dtype=torch.float32, device=cfg.device
            )

            # Weight by reward quality + how much this question's budget␣
↪differs from mean
            reward_weight = max(0.1, float(record["best_reward"]))
            deviation_weight = 1.0 + abs(target_budget - budget_mean) /␣
↪budget_std
            loss = reward_weight * deviation_weight * mse_loss(pred_norm,␣
↪target_norm).mean()

            optimizer.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(policy.parameters(), cfg.
↪grad_clip_norm)
            optimizer.step()

            losses.append(float(loss.item()))
            pred_budget = clamp_budget(
                denormalize_budget(pred_norm.item(), cfg.min_budget, cfg.
↪max_budget),
                cfg.min_budget, cfg.max_budget
            )
            pred_budgets.append(pred_budget)
            target_budgets.append(target_budget)

            if (step_idx + 1) % cfg.print_every == 0 or (step_idx + 1) ==␣
↪len(oracle_records):
                mae = np.mean(np.abs(np.array(pred_budgets) - np.
↪array(target_budgets)))
                print(f"[Warmstart {epoch+1}/{cfg.warmstart_epochs}] "
                      f"Step {step_idx+1}/{len(oracle_records)} | "
                      f"Loss={np.mean(losses):.4f} | "
                      f"PredAvg={np.mean(pred_budgets):.1f} | "
                      f"TargetAvg={np.mean(target_budgets):.1f} | MAE={mae:.
↪2f}")
```

```
        print(f"Warmstart epoch {epoch+1} done | Loss={np.mean(losses):.4f}")
        eval_det = evaluate_policy_deterministic(test_data, featurizer, policy,␣
↪llm, cfg)

        if eval_det["avg_reward"] > best_reward:
            best_reward = eval_det["avg_reward"]
            torch.save(policy.state_dict(), cfg.best_warmstart_path)
            print(f"  Saved best warmstart by reward = {best_reward:.4f}")

        if eval_det["accuracy"] > best_accuracy:
            best_accuracy = eval_det["accuracy"]
            torch.save(policy.state_dict(), cfg.best_warmstart_acc_path)
            print(f"  Saved best warmstart by accuracy = {best_accuracy:.4f}")


# ============================================================
# PPO Training
# This is the core fix over Model 5's basic policy gradient
#
# How PPO works here:
#   1. ROLLOUT: collect 300 (question, budget, reward) experiences
#              using current policy - all done with batched GPU calls
#   2. UPDATE:  run 4 epochs of mini-batch updates using the PPO clip
#              The clip prevents any single update from being too large
#   3. REPEAT:  for each RL epoch
#
# Why clip stops the oscillation:
#   - Old code: loss = -(log_prob * advantage) → unbounded update size
#   - PPO:      ratio = exp(new_log_prob - old_log_prob)
#               loss = -min(ratio*A, clip(ratio,0.8,1.2)*A)
#               If ratio > 1.2 or < 0.8, gradient is zeroed out
#               Policy cannot change by more than 20% per update
# ============================================================

def train_ppo(train_data, test_data, featurizer, policy, value_net,
              llm, cfg, oracle_records=None):
    print("\nStarting PPO training...")
    print(f"  PPO epsilon (clip range): {cfg.ppo_epsilon}")
    print(f"  PPO epochs per rollout: {cfg.ppo_epochs}")
    print(f"  Batch size: {cfg.llm_batch_size} questions at once on GPU")

    # Oracle lookup for blend signal
    oracle_lookup = {}
    if cfg.use_oracle_blend and oracle_records:
        for rec in oracle_records:
            oracle_lookup[rec["question"]] = rec["best_budget"]
```

```
    policy_optimizer = optim.Adam(policy.parameters(), lr=cfg.
↪policy_learning_rate)
    value_optimizer = optim.Adam(value_net.parameters(), lr=cfg.
↪value_learning_rate)
    mse_loss_fn = nn.MSELoss()

    questions = [item["question"] for item in train_data]
    gold_answers = [
        normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        for item in train_data
    ]

    # Pre-encode ALL training features once (saves time every epoch)
    print("  Pre-encoding all training features...")
    all_feats = featurizer.encode_batch(questions)
    all_feats_tensor = torch.tensor(all_feats, dtype=torch.float32, device=CFG.
↪device)

    best_test_reward = -1e9
    best_test_accuracy = -1.0

    for epoch in range(cfg.rl_epochs):
        print(f"\n{'='*80}")
        print(f"PPO Epoch {epoch+1}/{cfg.rl_epochs}")
        print(f"{'='*80}")

        # --------------------------------------------------------
        # PHASE 1: ROLLOUT - collect experiences
        # Sample budgets for all questions, run LLM, get rewards
        # All LLM calls are batched for GPU parallelism
        # --------------------------------------------------------
        print("  Phase 1: Collecting rollout experiences (batched)...")
        policy.eval()

        rollout_idx = list(range(len(questions)))
        random.shuffle(rollout_idx)

        # Sample budgets for ALL questions at once
        with torch.no_grad():
            norm_budgets, old_log_probs, _ = policy.
↪sample_budget(all_feats_tensor)
            norm_budgets = norm_budgets.squeeze(-1)
            old_log_probs = old_log_probs.detach()

        # Convert to actual budgets
        budget_list = []
```

```
        for i in range(len(questions)):
            nb = norm_budgets[i].item()
            b = clamp_budget(
                denormalize_budget(nb, cfg.min_budget, cfg.max_budget),
                cfg.min_budget, cfg.max_budget
            )
            # Warmup: mix in high budgets during first epochs
            if epoch < cfg.warmup_epochs_rl and random.random() < cfg.
↪warmup_mix_rl:
                b = random.randint(cfg.warmup_min_budget_high, cfg.max_budget)
            budget_list.append(b)

        # Group questions by budget, run batched LLM inference
        from collections import defaultdict
        budget_groups = defaultdict(list)
        for i, b in enumerate(budget_list):
            budget_groups[b].append(i)

        rewards_list = [None] * len(questions)
        corrects_list = [None] * len(questions)
        tokens_list = [None] * len(questions)

        for budget, indices in budget_groups.items():
            batch_q = [questions[i] for i in indices]
            batch_g = [gold_answers[i] for i in indices]
            batch_results = run_batch_two_stage(batch_q, batch_g, budget, llm,␣
↪cfg)
            for i, r in zip(indices, batch_results):
                rewards_list[i] = float(r["reward"])
                corrects_list[i] = int(r["correct"])
                tokens_list[i] = int(r["thinking_tokens_used"])

        rewards_tensor = torch.tensor(rewards_list, dtype=torch.float32,␣
↪device=cfg.device)

        # Compute advantages
        with torch.no_grad():
            values = value_net(all_feats_tensor)
            advantages = rewards_tensor - values
            # Normalize advantages for training stability
            advantages = (advantages - advantages.mean()) / (advantages.std() +␣
↪1e-8)

        print(f"  Rollout done | "
              f"Acc={np.mean(corrects_list):.4f} | "
              f"AvgReward={np.mean(rewards_list):.4f} | "
              f"AvgBudget={np.mean(budget_list):.1f} | "
```

```python
            f"StdBudget={np.std(budget_list):.1f}")

        # ---------------------------------------------------------
        # PHASE 2: PPO UPDATE - update policy using clip
        # Run ppo_epochs times on the same rollout data
        # Mini-batches shuffled each time for better gradient estimates
        # ---------------------------------------------------------
        print(f"  Phase 2: PPO updates ({cfg.ppo_epochs} epochs, batch={cfg.
↪ppo_batch_size})...")
        policy.train()

        norm_budgets_stored = norm_budgets.detach()

        for ppo_ep in range(cfg.ppo_epochs):
            indices = torch.randperm(len(questions))
            ep_policy_losses = []
            ep_value_losses = []
            ep_clip_fracs = []

            for start in range(0, len(questions), cfg.ppo_batch_size):
                batch_idx = indices[start : start + cfg.ppo_batch_size]

                batch_feats = all_feats_tensor[batch_idx]
                batch_old_log_probs = old_log_probs[batch_idx]
                batch_advantages = advantages[batch_idx]
                batch_rewards = rewards_tensor[batch_idx]
                batch_norm_budgets = norm_budgets_stored[batch_idx]

                # Get NEW log probs from updated policy
                new_log_probs, new_entropy = policy.get_log_prob(
                    batch_feats, batch_norm_budgets
                )

                # PPO ratio: how much has the policy changed?
                ratio = torch.exp(new_log_probs - batch_old_log_probs)

                # THE PPO CLIP - this is the fix for the oscillation
                # ratio > 1.2: policy is going too far in a good direction, cap␣
↪it
                # ratio < 0.8: policy is going too far in a bad direction, cap␣
↪it
                clipped_ratio = torch.clamp(
                    ratio,
                    1.0 - cfg.ppo_epsilon,
                    1.0 + cfg.ppo_epsilon
                )
```

```
                # Take the MINIMUM of clipped and unclipped
                # This is the pessimistic bound that prevents over-updating
                policy_loss = -torch.min(
                    ratio * batch_advantages,
                    clipped_ratio * batch_advantages
                ).mean()

                # Entropy bonus: keep policy from collapsing to one budget
                entropy_loss = -cfg.entropy_coef * new_entropy.mean()

                # Oracle blend: supervised signal on deterministic head
                # Keeps the deterministic head learning even during PPO
                supervised_loss = torch.tensor(0.0, device=cfg.device)
                if cfg.use_oracle_blend:
                    oracle_targets = []
                    valid_idx = []
                    for k, bi in enumerate(batch_idx.tolist()):
                        q = questions[bi]
                        if q in oracle_lookup:
                            oracle_targets.append(
                                budget_to_normalized(
                                    oracle_lookup[q], cfg.min_budget, cfg.max_budget
                                )
                            )
                            valid_idx.append(k)

                    if oracle_targets:
                        valid_feats = batch_feats[valid_idx]
                        oracle_norm_t = torch.tensor(
                            oracle_targets, dtype=torch.float32, device=cfg.device
                        ).unsqueeze(1)
                        det_pred = policy.deterministic_budget(valid_feats)
                        supervised_loss = nn.functional.mse_loss(det_pred, oracle_norm_t)

                total_policy_loss = (
                    policy_loss + entropy_loss + cfg.oracle_blend_weight * supervised_loss
                )

                policy_optimizer.zero_grad()
                total_policy_loss.backward()
                torch.nn.utils.clip_grad_norm_(policy.parameters(), cfg.grad_clip_norm)
                policy_optimizer.step()
```

```
                # Value network update
                value_preds = value_net(batch_feats)
                value_loss = mse_loss_fn(value_preds, batch_rewards)
                value_optimizer.zero_grad()
                (cfg.value_loss_coef * value_loss).backward()
                torch.nn.utils.clip_grad_norm_(value_net.parameters(), cfg.grad_clip_norm)
                value_optimizer.step()

                # Track clip fraction (diagnostic: how often does clip activate?)
                clip_frac = ((ratio - 1.0).abs() > cfg.ppo_epsilon).float().mean().item()
                ep_policy_losses.append(policy_loss.item())
                ep_value_losses.append(value_loss.item())
                ep_clip_fracs.append(clip_frac)

            print(f"    PPO update epoch {ppo_ep+1}/{cfg.ppo_epochs} | "
                  f"PolicyLoss={np.mean(ep_policy_losses):.4f} | "
                  f"ValueLoss={np.mean(ep_value_losses):.4f} | "
                  f"ClipFrac={np.mean(ep_clip_fracs):.3f}")
            # ClipFrac guide:
            #   ~0.0: updates too small, increase LR or decrease epsilon
            #   0.1-0.3: healthy range
            #   >0.5: updates too large, something is wrong

        # Evaluate after each RL epoch
        print("\n  Evaluating deterministic policy on test set...")
        eval_det = evaluate_policy_deterministic(test_data, featurizer, policy, llm, cfg)
        print(f"  Test Accuracy  : {eval_det['accuracy']:.4f}")
        print(f"  Test AvgTokens : {eval_det['avg_thinking_tokens']:.1f}")
        print(f"  Test AvgReward : {eval_det['avg_reward']:.4f}")
        print(f"  Test Budget    : {eval_det['avg_budget']:.1f} +/- "
              f"{eval_det['std_budget']:.1f} "
              f"[{eval_det['min_budget']:.0f}, {eval_det['max_budget']:.0f}]")

        if eval_det["avg_reward"] > best_test_reward:
            best_test_reward = eval_det["avg_reward"]
            torch.save(policy.state_dict(), cfg.best_ppo_path)
            torch.save(value_net.state_dict(), cfg.best_value_path)
            print(f"  Saved best PPO model by reward = {best_test_reward:.4f}")

        if eval_det["accuracy"] > best_test_accuracy:
            best_test_accuracy = eval_det["accuracy"]
            torch.save(policy.state_dict(), cfg.best_ppo_acc_path)
```

```python
            print(f"  Saved best PPO model by accuracy = {best_test_accuracy:.
↪4f}")


# ============================================================
# Main
# ============================================================

def main():
    set_seed(CFG.seed)

    print("torch version:", torch.__version__)
    print("cuda available:", torch.cuda.is_available())
    print("cuda device count:", torch.cuda.device_count())
    if torch.cuda.is_available():
        print("gpu name:", torch.cuda.get_device_name(0))
        print("gpu memory:", torch.cuda.get_device_properties(0).total_memory //
↪ 1e9, "GB")
    else:
        print("WARNING: No GPU detected. This will be very slow.")

    if CFG.clear_cache_on_start:
        print(f"\nClearing cache: {CFG.cache_dir}")
        reset_cache_dir(CFG.cache_dir)
        # Also remove any stale oracle cache from previous runs
        # This was the Model 6 bug: it loaded wrong oracle labels silently
        for stale in [CFG.oracle_cache_path,
                      "oracle_labels_v11.json", "oracle_labels_v12.json"]:
            if os.path.exists(stale):
                os.remove(stale)
                print(f"  Removed stale oracle cache: {stale}")
    else:
        ensure_dir(CFG.cache_dir)

    print(f"Using device: {CFG.device}")
    print(f"LLM batch size: {CFG.llm_batch_size} (questions processed in␣
↪parallel)")

    print("\nLoading GSM8K...")
    ds = load_dataset("gsm8k", "main")
    train_data = list(ds["train"].select(range(min(CFG.train_subset,␣
↪len(ds["train"])))))
    test_data = list(ds["test"].select(range(min(CFG.test_subset,␣
↪len(ds["test"])))))
    print(f"Train: {len(train_data)} | Test: {len(test_data)}")

    print("\nLoading featurizer...")
```

```python
    featurizer = QuestionFeaturizer(CFG.embed_name, CFG.device)
    input_dim = featurizer.encode(train_data[0]["question"]).shape[0]
    print(f"Feature dim: {input_dim}")

    print("\nLoading policy and value network...")
    policy = ContinuousBudgetPolicy(input_dim=input_dim).to(CFG.device)
    value_net = ValueNetwork(input_dim=input_dim).to(CFG.device)

    # Initialize toward 176 tokens (normalized ~0.72 for range 128-220)
    with torch.no_grad():
        policy.mean_head.bias.fill_(0.85)

    print("\nLoading LLM (with batched inference)...")
    llm = LLMReasoner(CFG.llm_name, CFG.device, batch_size=CFG.llm_batch_size)

    # Fixed budget baselines
    print("\n" + "="*70)
    print("FIXED BUDGET BASELINES")
    print("="*70)
    baseline_results = {}
    for b in CFG.baseline_budgets:
        res = evaluate_fixed_budget(test_data, llm, CFG, b)
        baseline_results[b] = res
        print(f"Fixed {b:>3} | Acc={res['accuracy']:.4f} | "
              f"AvgTokens={res['avg_thinking_tokens']:.0f} | "
              f"AvgReward={res['avg_reward']:.4f}")

    # Build oracle labels
    oracle_records = build_oracle_budget_labels(train_data, llm, CFG)

    # Warm-start
    warmstart_policy(oracle_records, test_data, featurizer, policy, llm, CFG)

    # Load best warm-start
    if os.path.exists(CFG.best_warmstart_acc_path):
        print("\nLoading best warm-start policy by accuracy...")
        policy.load_state_dict(torch.load(CFG.best_warmstart_acc_path,␣
↪map_location=CFG.device))
    elif os.path.exists(CFG.best_warmstart_path):
        print("\nLoading best warm-start policy by reward...")
        policy.load_state_dict(torch.load(CFG.best_warmstart_path,␣
↪map_location=CFG.device))

    print("\nPost warm-start evaluation:")
    warm_eval = evaluate_policy_deterministic(test_data, featurizer, policy,␣
↪llm, CFG)
    print(json.dumps({"warmstart_eval": warm_eval}, indent=2))
```

```
    # Pre-compute all training features (used for value pre-training and PPO)
    print("\nPre-computing all training features...")
    all_feats = featurizer.encode_batch([item["question"] for item in␣
↪train_data])
    all_feats_tensor = torch.tensor(all_feats, dtype=torch.float32, device=CFG.
↪device)

    # --------------------------------------------------------
    # FIX: Pre-train value network before PPO starts
    # Without this, ValueLoss starts at 2.6 (random init)
    # causing garbage advantage estimates in PPO epoch 1
    # Pre-training brings it down so epoch 1 updates are clean
    # --------------------------------------------------------
    print("\nPre-training value network on oracle rewards...")
    value_pretrain_optimizer = optim.Adam(value_net.parameters(), lr=CFG.
↪value_learning_rate)
    oracle_avg_reward = float(np.mean([r["best_reward"] for r in␣
↪oracle_records]))
    print(f"  Target oracle avg reward: {oracle_avg_reward:.4f}")

    value_net.train()
    for pretrain_ep in range(5):
        # Shuffle for better gradient estimates
        idx_perm = torch.randperm(len(train_data))
        ep_losses = []
        for start in range(0, len(train_data), 32):
            batch_idx = idx_perm[start:start+32]
            batch_feats = all_feats_tensor[batch_idx]

            # Per-question target: use oracle reward if available, else avg
            batch_targets = []
            for bi in batch_idx.tolist():
                q = train_data[bi]["question"]
                rec = next((r for r in oracle_records if r["question"] == q),␣
↪None)
                target = float(rec["best_reward"]) if rec else oracle_avg_reward
                batch_targets.append(target)

            target_t = torch.tensor(batch_targets, dtype=torch.float32,␣
↪device=CFG.device)
            preds = value_net(batch_feats)
            loss = nn.functional.mse_loss(preds, target_t)
            value_pretrain_optimizer.zero_grad()
            loss.backward()
```

```
            torch.nn.utils.clip_grad_norm_(value_net.parameters(), CFG.
↪grad_clip_norm)
            value_pretrain_optimizer.step()
            ep_losses.append(loss.item())

        print(f"  Value pretrain epoch {pretrain_ep+1}/5 | Loss={np.
↪mean(ep_losses):.4f}")

    print("Value pre-training complete. PPO will start with better advantage␣
↪estimates.\n")

    # PPO Training
    train_ppo(
        train_data=train_data,
        test_data=test_data,
        featurizer=featurizer,
        policy=policy,
        value_net=value_net,
        llm=llm,
        cfg=CFG,
        oracle_records=oracle_records,
    )

    # Load best PPO model
    print("\nLoading best PPO model for final evaluation...")
    if os.path.exists(CFG.best_ppo_acc_path):
        policy.load_state_dict(torch.load(CFG.best_ppo_acc_path,␣
↪map_location=CFG.device))
    elif os.path.exists(CFG.best_ppo_path):
        policy.load_state_dict(torch.load(CFG.best_ppo_path, map_location=CFG.
↪device))
    if os.path.exists(CFG.best_value_path):
        value_net.load_state_dict(torch.load(CFG.best_value_path,␣
↪map_location=CFG.device))

    print("\nFinal deterministic policy evaluation:")
    final_det = evaluate_policy_deterministic(test_data, featurizer, policy,␣
↪llm, CFG)
    print(json.dumps({"final_deterministic_eval": final_det}, indent=2))

    torch.save(policy.state_dict(), CFG.final_policy_path)
    torch.save(value_net.state_dict(), CFG.final_value_path)
    print(f"\nSaved: {CFG.final_policy_path}")
    print(f"Saved: {CFG.final_value_path}")

    # Final comparison table
```

```python
    print("\n" + "="*72)
    print("FINAL RESULTS COMPARISON")
    print("="*72)
    print(f"{'Method':<35} {'Acc':>6} {'AvgTokens':>10} {'AvgReward':>10}")
    print("-"*72)
    for b, res in baseline_results.items():
        print(f"  Fixed budget {b:<22} {res['accuracy']:>6.4f} "
              f"{res['avg_thinking_tokens']:>10.0f} {res['avg_reward']:>10.4f}")
    print(f"  Policy (PPO deterministic)     {final_det['accuracy']:>6.4f} "
          f"{final_det['avg_thinking_tokens']:>10.0f} {final_det['avg_reward']:
↪>10.4f}")
    print("="*72)
    print("\nAI Disclosure: Written with Claude's assistance")


main()
```

```
/opt/conda/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress
not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm

torch version: 2.11.0+cu128
cuda available: True
cuda device count: 1
gpu name: NVIDIA H200 NVL
gpu memory: 150.0 GB

Clearing cache: ./cache_ppo_v13

Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.

  Removed stale oracle cache: oracle_labels_v13.json
Using device: cuda
LLM batch size: 16 (questions processed in parallel)

Loading GSM8K…
Train: 300 | Test: 100

Loading featurizer…

Loading weights: 100%|      | 103/103 [00:00<00:00, 951.34it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture;
```

```
not ok if you expect identical arch.

Feature dim: 434

Loading policy and value network…

Loading LLM (with batched inference)…

`torch_dtype` is deprecated! Use `dtype` instead!
Loading weights: 100%|     | 338/338 [00:12<00:00, 26.85it/s]

======================================================================
FIXED BUDGET BASELINES
======================================================================

Evaluating fixed budget = 128…
  Fixed 128 | 80/100 | Acc=0.2500 | AvgReward=0.2485
  Fixed 128 | 100/100 | Acc=0.2500 | AvgReward=0.2485
Fixed 128 | Acc=0.2500 | AvgTokens=128 | AvgReward=0.2485

Evaluating fixed budget = 144…
  Fixed 144 | 80/100 | Acc=0.2125 | AvgReward=0.1936
  Fixed 144 | 100/100 | Acc=0.2400 | AvgReward=0.2280
Fixed 144 | Acc=0.2400 | AvgTokens=144 | AvgReward=0.2280

Evaluating fixed budget = 160…
  Fixed 160 | 80/100 | Acc=0.2250 | AvgReward=0.2012
  Fixed 160 | 100/100 | Acc=0.2500 | AvgReward=0.2325
Fixed 160 | Acc=0.2500 | AvgTokens=160 | AvgReward=0.2325

Evaluating fixed budget = 168…
  Fixed 168 | 80/100 | Acc=0.2875 | AvgReward=0.2754
  Fixed 168 | 100/100 | Acc=0.2900 | AvgReward=0.2785
Fixed 168 | Acc=0.2900 | AvgTokens=168 | AvgReward=0.2785

Evaluating fixed budget = 176…
  Fixed 176 | 80/100 | Acc=0.3500 | AvgReward=0.3495
  Fixed 176 | 100/100 | Acc=0.3500 | AvgReward=0.3495
Fixed 176 | Acc=0.3500 | AvgTokens=176 | AvgReward=0.3495

Evaluating fixed budget = 184…
  Fixed 184 | 80/100 | Acc=0.3875 | AvgReward=0.3924
  Fixed 184 | 100/100 | Acc=0.3900 | AvgReward=0.3955
Fixed 184 | Acc=0.3900 | AvgTokens=184 | AvgReward=0.3955

Evaluating fixed budget = 192…
  Fixed 192 | 80/100 | Acc=0.4000 | AvgReward=0.4040
  Fixed 192 | 100/100 | Acc=0.4100 | AvgReward=0.4165
```

```
Fixed 192 | Acc=0.4100 | AvgTokens=192 | AvgReward=0.4165

Evaluating fixed budget = 210…
  Fixed 210 | 80/100 | Acc=0.4250 | AvgReward=0.4262
  Fixed 210 | 100/100 | Acc=0.4500 | AvgReward=0.4575
Fixed 210 | Acc=0.4500 | AvgTokens=210 | AvgReward=0.4575

Evaluating fixed budget = 220…
  Fixed 220 | 80/100 | Acc=0.4125 | AvgReward=0.4056
  Fixed 220 | 100/100 | Acc=0.4200 | AvgReward=0.4150
Fixed 220 | Acc=0.4200 | AvgTokens=220 | AvgReward=0.4150

Building oracle labels with BATCHED GPU inference…
  Trying budget=128 for all 300 questions…
    Budget 128 done in 115.7s | Correct: 69/300 (23.0%)
  Trying budget=144 for all 300 questions…
    Budget 144 done in 72.3s | Correct: 77/300 (25.7%)
  Trying budget=160 for all 300 questions…
    Budget 160 done in 77.7s | Correct: 90/300 (30.0%)
  Trying budget=168 for all 300 questions…
    Budget 168 done in 78.2s | Correct: 97/300 (32.3%)
  Trying budget=176 for all 300 questions…
    Budget 176 done in 86.8s | Correct: 103/300 (34.3%)
  Trying budget=184 for all 300 questions…
    Budget 184 done in 85.1s | Correct: 112/300 (37.3%)
  Trying budget=192 for all 300 questions…
    Budget 192 done in 88.2s | Correct: 123/300 (41.0%)
  Trying budget=200 for all 300 questions…
    Budget 200 done in 93.6s | Correct: 128/300 (42.7%)
  Trying budget=210 for all 300 questions…
    Budget 210 done in 98.7s | Correct: 139/300 (46.3%)
  Trying budget=220 for all 300 questions…
    Budget 220 done in 105.2s | Correct: 138/300 (46.0%)

Oracle stats: mean=144.2, std=26.6, solvable=0.573
  budget=128: 197 questions (65.7%)
  budget=144: 22 questions (7.3%)
  budget=160: 18 questions (6.0%)
  budget=168: 15 questions (5.0%)
  budget=176: 7 questions (2.3%)
  budget=184: 9 questions (3.0%)
  budget=192: 8 questions (2.7%)
  budget=200: 5 questions (1.7%)
  budget=210: 12 questions (4.0%)
  budget=220: 7 questions (2.3%)
Saved oracle labels to oracle_labels_v13.json

Starting supervised warm-start…
```

```
[Warmstart 1/15] Step 20/300 | Loss=0.2799 | PredAvg=138.0 | TargetAvg=139.7 |
MAE=18.80
[Warmstart 1/15] Step 40/300 | Loss=0.3242 | PredAvg=135.3 | TargetAvg=141.7 |
MAE=18.40
[Warmstart 1/15] Step 60/300 | Loss=0.3281 | PredAvg=134.8 | TargetAvg=141.3 |
MAE=18.05
[Warmstart 1/15] Step 80/300 | Loss=0.4423 | PredAvg=133.9 | TargetAvg=143.8 |
MAE=19.80
[Warmstart 1/15] Step 100/300 | Loss=0.3714 | PredAvg=137.1 | TargetAvg=143.5 |
MAE=19.57
[Warmstart 1/15] Step 120/300 | Loss=0.4068 | PredAvg=138.3 | TargetAvg=146.1 |
MAE=21.77
[Warmstart 1/15] Step 140/300 | Loss=0.3756 | PredAvg=139.5 | TargetAvg=145.2 |
MAE=21.92
[Warmstart 1/15] Step 160/300 | Loss=0.3517 | PredAvg=138.5 | TargetAvg=144.6 |
MAE=20.86
[Warmstart 1/15] Step 180/300 | Loss=0.3635 | PredAvg=139.4 | TargetAvg=145.3 |
MAE=21.82
[Warmstart 1/15] Step 200/300 | Loss=0.3440 | PredAvg=139.2 | TargetAvg=144.7 |
MAE=21.50
[Warmstart 1/15] Step 220/300 | Loss=0.3572 | PredAvg=139.2 | TargetAvg=145.0 |
MAE=21.51
[Warmstart 1/15] Step 240/300 | Loss=0.3362 | PredAvg=138.8 | TargetAvg=144.1 |
MAE=20.77
[Warmstart 1/15] Step 260/300 | Loss=0.3215 | PredAvg=138.3 | TargetAvg=144.0 |
MAE=20.35
[Warmstart 1/15] Step 280/300 | Loss=0.3373 | PredAvg=137.9 | TargetAvg=144.0 |
MAE=20.38
[Warmstart 1/15] Step 300/300 | Loss=0.3465 | PredAvg=138.0 | TargetAvg=144.2 |
MAE=20.63
Warmstart epoch 1 done | Loss=0.3465
  Det eval complete | Acc=0.2700 | AvgBudget=146.7 | StdBudget=10.8 |
Range=[129, 171]
  Saved best warmstart by reward = 0.2641
  Saved best warmstart by accuracy = 0.2700
[Warmstart 2/15] Step 20/300 | Loss=0.1157 | PredAvg=142.8 | TargetAvg=138.0 |
MAE=16.60
[Warmstart 2/15] Step 40/300 | Loss=0.2151 | PredAvg=141.2 | TargetAvg=143.2 |
MAE=17.95
[Warmstart 2/15] Step 60/300 | Loss=0.2523 | PredAvg=145.6 | TargetAvg=146.0 |
MAE=22.67
[Warmstart 2/15] Step 80/300 | Loss=0.2642 | PredAvg=145.3 | TargetAvg=144.6 |
MAE=22.77
[Warmstart 2/15] Step 100/300 | Loss=0.3140 | PredAvg=143.1 | TargetAvg=145.8 |
MAE=22.62
[Warmstart 2/15] Step 120/300 | Loss=0.3016 | PredAvg=145.0 | TargetAvg=146.0 |
MAE=23.43
[Warmstart 2/15] Step 140/300 | Loss=0.2852 | PredAvg=145.5 | TargetAvg=144.7 |
```

```
MAE=23.01
[Warmstart 2/15] Step 160/300 | Loss=0.2767 | PredAvg=143.9 | TargetAvg=144.1 |
MAE=22.06
[Warmstart 2/15] Step 180/300 | Loss=0.2589 | PredAvg=142.3 | TargetAvg=143.1 |
MAE=20.32
[Warmstart 2/15] Step 200/300 | Loss=0.2725 | PredAvg=141.4 | TargetAvg=143.1 |
MAE=20.20
[Warmstart 2/15] Step 220/300 | Loss=0.2681 | PredAvg=140.8 | TargetAvg=143.0 |
MAE=19.70
[Warmstart 2/15] Step 240/300 | Loss=0.2665 | PredAvg=140.5 | TargetAvg=142.8 |
MAE=19.39
[Warmstart 2/15] Step 260/300 | Loss=0.2818 | PredAvg=141.2 | TargetAvg=143.2 |
MAE=20.08
[Warmstart 2/15] Step 280/300 | Loss=0.2997 | PredAvg=141.2 | TargetAvg=143.8 |
MAE=20.31
[Warmstart 2/15] Step 300/300 | Loss=0.3064 | PredAvg=141.0 | TargetAvg=144.2 |
MAE=20.71
Warmstart epoch 2 done | Loss=0.3064
  Det eval complete | Acc=0.2900 | AvgBudget=147.8 | StdBudget=10.6 |
Range=[129, 171]
  Saved best warmstart by reward = 0.2886
  Saved best warmstart by accuracy = 0.2900
[Warmstart 3/15] Step 20/300 | Loss=0.1387 | PredAvg=153.4 | TargetAvg=140.8 |
MAE=25.85
[Warmstart 3/15] Step 40/300 | Loss=0.2165 | PredAvg=147.0 | TargetAvg=143.7 |
MAE=24.10
[Warmstart 3/15] Step 60/300 | Loss=0.2155 | PredAvg=146.1 | TargetAvg=144.3 |
MAE=23.03
[Warmstart 3/15] Step 80/300 | Loss=0.2139 | PredAvg=145.3 | TargetAvg=143.3 |
MAE=22.61
[Warmstart 3/15] Step 100/300 | Loss=0.2453 | PredAvg=144.0 | TargetAvg=143.9 |
MAE=22.83
[Warmstart 3/15] Step 120/300 | Loss=0.2187 | PredAvg=142.5 | TargetAvg=141.9 |
MAE=20.60
[Warmstart 3/15] Step 140/300 | Loss=0.2228 | PredAvg=141.0 | TargetAvg=142.0 |
MAE=19.84
[Warmstart 3/15] Step 160/300 | Loss=0.2132 | PredAvg=140.4 | TargetAvg=141.9 |
MAE=19.50
[Warmstart 3/15] Step 180/300 | Loss=0.2495 | PredAvg=139.9 | TargetAvg=142.2 |
MAE=19.67
[Warmstart 3/15] Step 200/300 | Loss=0.3008 | PredAvg=140.2 | TargetAvg=143.6 |
MAE=20.71
[Warmstart 3/15] Step 220/300 | Loss=0.2939 | PredAvg=141.0 | TargetAvg=143.5 |
MAE=20.90
[Warmstart 3/15] Step 240/300 | Loss=0.2824 | PredAvg=141.0 | TargetAvg=143.6 |
MAE=20.64
[Warmstart 3/15] Step 260/300 | Loss=0.3062 | PredAvg=140.6 | TargetAvg=144.3 |
MAE=21.03
```

```
[Warmstart 3/15] Step 280/300 | Loss=0.3043 | PredAvg=141.5 | TargetAvg=144.4 |
MAE=21.49
[Warmstart 3/15] Step 300/300 | Loss=0.3086 | PredAvg=141.5 | TargetAvg=144.2 |
MAE=21.47
Warmstart epoch 3 done | Loss=0.3086
  Det eval complete | Acc=0.2300 | AvgBudget=142.6 | StdBudget=9.3 | Range=[128,
165]
[Warmstart 4/15] Step 20/300 | Loss=0.1948 | PredAvg=137.8 | TargetAvg=139.7 |
MAE=15.45
[Warmstart 4/15] Step 40/300 | Loss=0.2499 | PredAvg=137.6 | TargetAvg=140.6 |
MAE=15.57
[Warmstart 4/15] Step 60/300 | Loss=0.2483 | PredAvg=139.4 | TargetAvg=139.9 |
MAE=17.58
[Warmstart 4/15] Step 80/300 | Loss=0.2567 | PredAvg=138.9 | TargetAvg=141.0 |
MAE=17.98
[Warmstart 4/15] Step 100/300 | Loss=0.3175 | PredAvg=138.6 | TargetAvg=143.9 |
MAE=19.36
[Warmstart 4/15] Step 120/300 | Loss=0.3349 | PredAvg=140.7 | TargetAvg=145.2 |
MAE=20.98
[Warmstart 4/15] Step 140/300 | Loss=0.3226 | PredAvg=142.2 | TargetAvg=144.7 |
MAE=21.81
[Warmstart 4/15] Step 160/300 | Loss=0.3229 | PredAvg=142.8 | TargetAvg=145.2 |
MAE=22.44
[Warmstart 4/15] Step 180/300 | Loss=0.3261 | PredAvg=142.5 | TargetAvg=144.5 |
MAE=21.89
[Warmstart 4/15] Step 200/300 | Loss=0.3003 | PredAvg=141.4 | TargetAvg=143.8 |
MAE=20.63
[Warmstart 4/15] Step 220/300 | Loss=0.3182 | PredAvg=141.2 | TargetAvg=144.2 |
MAE=21.18
[Warmstart 4/15] Step 240/300 | Loss=0.3096 | PredAvg=141.0 | TargetAvg=143.9 |
MAE=20.84
[Warmstart 4/15] Step 260/300 | Loss=0.3300 | PredAvg=140.5 | TargetAvg=144.7 |
MAE=21.25
[Warmstart 4/15] Step 280/300 | Loss=0.3195 | PredAvg=141.2 | TargetAvg=144.9 |
MAE=21.61
[Warmstart 4/15] Step 300/300 | Loss=0.3004 | PredAvg=141.4 | TargetAvg=144.2 |
MAE=21.18
Warmstart epoch 4 done | Loss=0.3004
  Det eval complete | Acc=0.2500 | AvgBudget=140.2 | StdBudget=8.3 | Range=[128,
162]
[Warmstart 5/15] Step 20/300 | Loss=0.1401 | PredAvg=144.4 | TargetAvg=142.8 |
MAE=17.60
[Warmstart 5/15] Step 40/300 | Loss=0.1605 | PredAvg=139.5 | TargetAvg=140.4 |
MAE=16.18
[Warmstart 5/15] Step 60/300 | Loss=0.3584 | PredAvg=139.5 | TargetAvg=146.3 |
MAE=20.83
[Warmstart 5/15] Step 80/300 | Loss=0.3315 | PredAvg=139.6 | TargetAvg=146.1 |
MAE=20.86
```

```
[Warmstart 5/15] Step 100/300 | Loss=0.2970 | PredAvg=140.5 | TargetAvg=144.9 |
MAE=20.67
[Warmstart 5/15] Step 120/300 | Loss=0.3152 | PredAvg=139.7 | TargetAvg=145.1 |
MAE=20.61
[Warmstart 5/15] Step 140/300 | Loss=0.3033 | PredAvg=139.8 | TargetAvg=144.7 |
MAE=20.45
[Warmstart 5/15] Step 160/300 | Loss=0.3182 | PredAvg=140.4 | TargetAvg=144.4 |
MAE=20.95
[Warmstart 5/15] Step 180/300 | Loss=0.3429 | PredAvg=140.7 | TargetAvg=145.2 |
MAE=21.90
[Warmstart 5/15] Step 200/300 | Loss=0.3311 | PredAvg=140.8 | TargetAvg=144.9 |
MAE=21.71
[Warmstart 5/15] Step 220/300 | Loss=0.3468 | PredAvg=141.9 | TargetAvg=145.8 |
MAE=22.42
[Warmstart 5/15] Step 240/300 | Loss=0.3348 | PredAvg=142.1 | TargetAvg=145.4 |
MAE=22.43
[Warmstart 5/15] Step 260/300 | Loss=0.3123 | PredAvg=142.1 | TargetAvg=144.5 |
MAE=21.68
[Warmstart 5/15] Step 280/300 | Loss=0.3042 | PredAvg=141.4 | TargetAvg=144.1 |
MAE=21.11
[Warmstart 5/15] Step 300/300 | Loss=0.3028 | PredAvg=140.8 | TargetAvg=144.2 |
MAE=20.90
Warmstart epoch 5 done | Loss=0.3028
  Det eval complete | Acc=0.2400 | AvgBudget=138.9 | StdBudget=7.5 | Range=[128,
159]
[Warmstart 6/15] Step 20/300 | Loss=0.3077 | PredAvg=138.2 | TargetAvg=141.4 |
MAE=19.15
[Warmstart 6/15] Step 40/300 | Loss=0.2437 | PredAvg=139.3 | TargetAvg=142.3 |
MAE=19.93
[Warmstart 6/15] Step 60/300 | Loss=0.2140 | PredAvg=139.1 | TargetAvg=142.7 |
MAE=19.10
[Warmstart 6/15] Step 80/300 | Loss=0.2872 | PredAvg=140.6 | TargetAvg=145.0 |
MAE=20.80
[Warmstart 6/15] Step 100/300 | Loss=0.2750 | PredAvg=140.0 | TargetAvg=143.1 |
MAE=19.58
[Warmstart 6/15] Step 120/300 | Loss=0.2928 | PredAvg=139.5 | TargetAvg=143.8 |
MAE=19.60
[Warmstart 6/15] Step 140/300 | Loss=0.2686 | PredAvg=140.0 | TargetAvg=143.0 |
MAE=19.19
[Warmstart 6/15] Step 160/300 | Loss=0.3023 | PredAvg=139.3 | TargetAvg=143.5 |
MAE=19.62
[Warmstart 6/15] Step 180/300 | Loss=0.3210 | PredAvg=139.4 | TargetAvg=144.6 |
MAE=20.38
[Warmstart 6/15] Step 200/300 | Loss=0.3374 | PredAvg=139.8 | TargetAvg=145.2 |
MAE=20.89
[Warmstart 6/15] Step 220/300 | Loss=0.3284 | PredAvg=139.7 | TargetAvg=144.8 |
MAE=20.76
[Warmstart 6/15] Step 240/300 | Loss=0.3145 | PredAvg=139.6 | TargetAvg=144.2 |
```

```
MAE=20.62
[Warmstart 6/15] Step 260/300 | Loss=0.3179 | PredAvg=139.5 | TargetAvg=144.2 |
MAE=20.46
[Warmstart 6/15] Step 280/300 | Loss=0.3160 | PredAvg=139.5 | TargetAvg=144.1 |
MAE=20.32
[Warmstart 6/15] Step 300/300 | Loss=0.3027 | PredAvg=139.5 | TargetAvg=144.2 |
MAE=20.02
Warmstart epoch 6 done | Loss=0.3027
  Det eval complete | Acc=0.2500 | AvgBudget=144.8 | StdBudget=8.2 | Range=[129,
162]
[Warmstart 7/15] Step 20/300 | Loss=0.0911 | PredAvg=154.3 | TargetAvg=145.2 |
MAE=19.15
[Warmstart 7/15] Step 40/300 | Loss=0.1157 | PredAvg=148.9 | TargetAvg=142.1 |
MAE=18.00
[Warmstart 7/15] Step 60/300 | Loss=0.2093 | PredAvg=144.2 | TargetAvg=142.5 |
MAE=18.62
[Warmstart 7/15] Step 80/300 | Loss=0.2014 | PredAvg=141.9 | TargetAvg=142.3 |
MAE=18.11
[Warmstart 7/15] Step 100/300 | Loss=0.2585 | PredAvg=141.2 | TargetAvg=143.4 |
MAE=18.93
[Warmstart 7/15] Step 120/300 | Loss=0.2982 | PredAvg=141.1 | TargetAvg=143.9 |
MAE=19.86
[Warmstart 7/15] Step 140/300 | Loss=0.3249 | PredAvg=140.6 | TargetAvg=145.0 |
MAE=20.46
[Warmstart 7/15] Step 160/300 | Loss=0.2994 | PredAvg=140.9 | TargetAvg=144.3 |
MAE=20.24
[Warmstart 7/15] Step 180/300 | Loss=0.3128 | PredAvg=141.1 | TargetAvg=145.1 |
MAE=20.97
[Warmstart 7/15] Step 200/300 | Loss=0.2939 | PredAvg=141.1 | TargetAvg=144.5 |
MAE=20.50
[Warmstart 7/15] Step 220/300 | Loss=0.2820 | PredAvg=140.7 | TargetAvg=143.8 |
MAE=19.56
[Warmstart 7/15] Step 240/300 | Loss=0.2805 | PredAvg=140.4 | TargetAvg=144.5 |
MAE=19.72
[Warmstart 7/15] Step 260/300 | Loss=0.2662 | PredAvg=141.1 | TargetAvg=143.5 |
MAE=20.00
[Warmstart 7/15] Step 280/300 | Loss=0.2695 | PredAvg=140.6 | TargetAvg=143.3 |
MAE=19.65
[Warmstart 7/15] Step 300/300 | Loss=0.3048 | PredAvg=140.3 | TargetAvg=144.2 |
MAE=20.41
Warmstart epoch 7 done | Loss=0.3048
  Det eval complete | Acc=0.2200 | AvgBudget=138.1 | StdBudget=6.5 | Range=[128,
155]
[Warmstart 8/15] Step 20/300 | Loss=0.4108 | PredAvg=139.6 | TargetAvg=147.0 |
MAE=24.75
[Warmstart 8/15] Step 40/300 | Loss=0.2588 | PredAvg=140.5 | TargetAvg=145.5 |
MAE=20.45
[Warmstart 8/15] Step 60/300 | Loss=0.1842 | PredAvg=139.9 | TargetAvg=141.3 |
```

```
MAE=17.67
[Warmstart 8/15] Step 80/300 | Loss=0.2228 | PredAvg=138.5 | TargetAvg=142.6 |
MAE=18.02
[Warmstart 8/15] Step 100/300 | Loss=0.2730 | PredAvg=138.5 | TargetAvg=143.4 |
MAE=18.54
[Warmstart 8/15] Step 120/300 | Loss=0.2895 | PredAvg=139.2 | TargetAvg=143.2 |
MAE=19.17
[Warmstart 8/15] Step 140/300 | Loss=0.2767 | PredAvg=139.6 | TargetAvg=143.4 |
MAE=19.17
[Warmstart 8/15] Step 160/300 | Loss=0.2752 | PredAvg=140.2 | TargetAvg=143.6 |
MAE=19.61
[Warmstart 8/15] Step 180/300 | Loss=0.3148 | PredAvg=139.9 | TargetAvg=144.3 |
MAE=20.33
[Warmstart 8/15] Step 200/300 | Loss=0.3253 | PredAvg=139.5 | TargetAvg=144.6 |
MAE=20.53
[Warmstart 8/15] Step 220/300 | Loss=0.3187 | PredAvg=139.1 | TargetAvg=144.0 |
MAE=20.02
[Warmstart 8/15] Step 240/300 | Loss=0.3039 | PredAvg=138.5 | TargetAvg=143.7 |
MAE=19.50
[Warmstart 8/15] Step 260/300 | Loss=0.2849 | PredAvg=138.6 | TargetAvg=143.2 |
MAE=18.97
[Warmstart 8/15] Step 280/300 | Loss=0.2762 | PredAvg=138.3 | TargetAvg=143.0 |
MAE=18.62
[Warmstart 8/15] Step 300/300 | Loss=0.2992 | PredAvg=138.4 | TargetAvg=144.2 |
MAE=19.46
Warmstart epoch 8 done | Loss=0.2992
  Det eval complete | Acc=0.2500 | AvgBudget=148.1 | StdBudget=7.6 | Range=[131,
165]
[Warmstart 9/15] Step 20/300 | Loss=0.1825 | PredAvg=148.9 | TargetAvg=142.4 |
MAE=25.50
[Warmstart 9/15] Step 40/300 | Loss=0.1752 | PredAvg=143.9 | TargetAvg=141.7 |
MAE=20.75
[Warmstart 9/15] Step 60/300 | Loss=0.1345 | PredAvg=141.1 | TargetAvg=139.9 |
MAE=17.43
[Warmstart 9/15] Step 80/300 | Loss=0.2474 | PredAvg=139.5 | TargetAvg=143.1 |
MAE=19.43
[Warmstart 9/15] Step 100/300 | Loss=0.2030 | PredAvg=139.7 | TargetAvg=141.3 |
MAE=17.97
[Warmstart 9/15] Step 120/300 | Loss=0.2152 | PredAvg=139.1 | TargetAvg=141.3 |
MAE=17.56
[Warmstart 9/15] Step 140/300 | Loss=0.2138 | PredAvg=139.1 | TargetAvg=141.4 |
MAE=17.67
[Warmstart 9/15] Step 160/300 | Loss=0.2740 | PredAvg=138.7 | TargetAvg=142.7 |
MAE=18.89
[Warmstart 9/15] Step 180/300 | Loss=0.2508 | PredAvg=138.9 | TargetAvg=141.9 |
MAE=18.37
[Warmstart 9/15] Step 200/300 | Loss=0.2384 | PredAvg=138.2 | TargetAvg=141.7 |
MAE=17.82
```

```
[Warmstart 9/15] Step 220/300 | Loss=0.2735 | PredAvg=138.2 | TargetAvg=143.0 |
MAE=18.72
[Warmstart 9/15] Step 240/300 | Loss=0.2684 | PredAvg=138.8 | TargetAvg=143.6 |
MAE=18.85
[Warmstart 9/15] Step 260/300 | Loss=0.2689 | PredAvg=139.6 | TargetAvg=143.7 |
MAE=19.27
[Warmstart 9/15] Step 280/300 | Loss=0.2689 | PredAvg=139.5 | TargetAvg=143.7 |
MAE=19.16
[Warmstart 9/15] Step 300/300 | Loss=0.2925 | PredAvg=139.5 | TargetAvg=144.2 |
MAE=19.59
Warmstart epoch 9 done | Loss=0.2925
  Det eval complete | Acc=0.2400 | AvgBudget=142.2 | StdBudget=6.5 | Range=[130,
159]
[Warmstart 10/15] Step 20/300 | Loss=0.0693 | PredAvg=136.3 | TargetAvg=136.0 |
MAE=12.55
[Warmstart 10/15] Step 40/300 | Loss=0.1186 | PredAvg=137.2 | TargetAvg=138.0 |
MAE=14.10
[Warmstart 10/15] Step 60/300 | Loss=0.3228 | PredAvg=136.2 | TargetAvg=142.2 |
MAE=17.50
[Warmstart 10/15] Step 80/300 | Loss=0.3241 | PredAvg=137.0 | TargetAvg=142.6 |
MAE=18.52
[Warmstart 10/15] Step 100/300 | Loss=0.3860 | PredAvg=137.5 | TargetAvg=145.1 |
MAE=20.68
[Warmstart 10/15] Step 120/300 | Loss=0.3819 | PredAvg=139.4 | TargetAvg=146.3 |
MAE=21.66
[Warmstart 10/15] Step 140/300 | Loss=0.3574 | PredAvg=139.4 | TargetAvg=145.3 |
MAE=20.93
[Warmstart 10/15] Step 160/300 | Loss=0.3684 | PredAvg=138.6 | TargetAvg=145.7 |
MAE=21.02
[Warmstart 10/15] Step 180/300 | Loss=0.3533 | PredAvg=138.6 | TargetAvg=146.0 |
MAE=20.78
[Warmstart 10/15] Step 200/300 | Loss=0.3269 | PredAvg=139.1 | TargetAvg=145.2 |
MAE=20.45
[Warmstart 10/15] Step 220/300 | Loss=0.3149 | PredAvg=138.9 | TargetAvg=144.8 |
MAE=19.90
[Warmstart 10/15] Step 240/300 | Loss=0.2965 | PredAvg=138.6 | TargetAvg=144.2 |
MAE=19.40
[Warmstart 10/15] Step 260/300 | Loss=0.2873 | PredAvg=138.4 | TargetAvg=144.0 |
MAE=18.95
[Warmstart 10/15] Step 280/300 | Loss=0.2917 | PredAvg=138.5 | TargetAvg=144.3 |
MAE=19.05
[Warmstart 10/15] Step 300/300 | Loss=0.2915 | PredAvg=138.7 | TargetAvg=144.2 |
MAE=19.17
Warmstart epoch 10 done | Loss=0.2915
  Det eval complete | Acc=0.2500 | AvgBudget=139.2 | StdBudget=5.6 | Range=[130,
160]
[Warmstart 11/15] Step 20/300 | Loss=0.4298 | PredAvg=136.7 | TargetAvg=149.1 |
MAE=20.05
```

```
[Warmstart 11/15] Step 40/300 | Loss=0.3657 | PredAvg=135.1 | TargetAvg=145.4 |
MAE=18.20
[Warmstart 11/15] Step 60/300 | Loss=0.2766 | PredAvg=136.9 | TargetAvg=143.9 |
MAE=17.58
[Warmstart 11/15] Step 80/300 | Loss=0.3497 | PredAvg=137.3 | TargetAvg=146.1 |
MAE=19.60
[Warmstart 11/15] Step 100/300 | Loss=0.3282 | PredAvg=138.5 | TargetAvg=145.7 |
MAE=19.92
[Warmstart 11/15] Step 120/300 | Loss=0.3017 | PredAvg=138.8 | TargetAvg=145.4 |
MAE=19.38
[Warmstart 11/15] Step 140/300 | Loss=0.2924 | PredAvg=139.6 | TargetAvg=145.5 |
MAE=19.90
[Warmstart 11/15] Step 160/300 | Loss=0.2886 | PredAvg=139.2 | TargetAvg=145.0 |
MAE=19.41
[Warmstart 11/15] Step 180/300 | Loss=0.2599 | PredAvg=138.7 | TargetAvg=143.7 |
MAE=18.21
[Warmstart 11/15] Step 200/300 | Loss=0.2642 | PredAvg=138.1 | TargetAvg=143.6 |
MAE=17.91
[Warmstart 11/15] Step 220/300 | Loss=0.2681 | PredAvg=138.0 | TargetAvg=143.6 |
MAE=17.89
[Warmstart 11/15] Step 240/300 | Loss=0.2804 | PredAvg=138.6 | TargetAvg=144.4 |
MAE=18.28
[Warmstart 11/15] Step 260/300 | Loss=0.2807 | PredAvg=139.2 | TargetAvg=144.8 |
MAE=18.69
[Warmstart 11/15] Step 280/300 | Loss=0.2796 | PredAvg=138.9 | TargetAvg=144.5 |
MAE=18.52
[Warmstart 11/15] Step 300/300 | Loss=0.2769 | PredAvg=138.5 | TargetAvg=144.2 |
MAE=18.21
Warmstart epoch 11 done | Loss=0.2769
  Det eval complete | Acc=0.2800 | AvgBudget=131.7 | StdBudget=3.0 | Range=[128,
146]
[Warmstart 12/15] Step 20/300 | Loss=0.4206 | PredAvg=132.1 | TargetAvg=149.3 |
MAE=21.40
[Warmstart 12/15] Step 40/300 | Loss=0.4181 | PredAvg=139.3 | TargetAvg=152.1 |
MAE=22.65
[Warmstart 12/15] Step 60/300 | Loss=0.3782 | PredAvg=141.3 | TargetAvg=148.9 |
MAE=22.60
[Warmstart 12/15] Step 80/300 | Loss=0.3514 | PredAvg=139.3 | TargetAvg=147.2 |
MAE=21.07
[Warmstart 12/15] Step 100/300 | Loss=0.3018 | PredAvg=138.3 | TargetAvg=145.9 |
MAE=19.62
[Warmstart 12/15] Step 120/300 | Loss=0.2834 | PredAvg=138.6 | TargetAvg=145.6 |
MAE=19.42
[Warmstart 12/15] Step 140/300 | Loss=0.2687 | PredAvg=138.1 | TargetAvg=144.8 |
MAE=18.58
[Warmstart 12/15] Step 160/300 | Loss=0.2702 | PredAvg=137.3 | TargetAvg=144.5 |
MAE=18.13
[Warmstart 12/15] Step 180/300 | Loss=0.2561 | PredAvg=137.1 | TargetAvg=144.0 |
```

```
MAE=17.63
[Warmstart 12/15] Step 200/300 | Loss=0.2837 | PredAvg=137.6 | TargetAvg=145.0 |
MAE=18.07
[Warmstart 12/15] Step 220/300 | Loss=0.2602 | PredAvg=138.0 | TargetAvg=144.2 |
MAE=17.47
[Warmstart 12/15] Step 240/300 | Loss=0.2562 | PredAvg=137.4 | TargetAvg=143.5 |
MAE=16.82
[Warmstart 12/15] Step 260/300 | Loss=0.2733 | PredAvg=136.9 | TargetAvg=143.8 |
MAE=17.02
[Warmstart 12/15] Step 280/300 | Loss=0.2721 | PredAvg=137.1 | TargetAvg=144.0 |
MAE=17.17
[Warmstart 12/15] Step 300/300 | Loss=0.2862 | PredAvg=137.2 | TargetAvg=144.2 |
MAE=17.57
Warmstart epoch 12 done | Loss=0.2862
  Det eval complete | Acc=0.2200 | AvgBudget=134.9 | StdBudget=4.7 | Range=[129,
155]
[Warmstart 13/15] Step 20/300 | Loss=0.1487 | PredAvg=143.4 | TargetAvg=145.7 |
MAE=19.70
[Warmstart 13/15] Step 40/300 | Loss=0.2023 | PredAvg=141.6 | TargetAvg=143.6 |
MAE=18.90
[Warmstart 13/15] Step 60/300 | Loss=0.1754 | PredAvg=138.2 | TargetAvg=141.2 |
MAE=15.87
[Warmstart 13/15] Step 80/300 | Loss=0.2860 | PredAvg=137.7 | TargetAvg=143.4 |
MAE=17.90
[Warmstart 13/15] Step 100/300 | Loss=0.2932 | PredAvg=138.2 | TargetAvg=145.2 |
MAE=18.95
[Warmstart 13/15] Step 120/300 | Loss=0.2560 | PredAvg=138.4 | TargetAvg=144.1 |
MAE=18.25
[Warmstart 13/15] Step 140/300 | Loss=0.2490 | PredAvg=137.6 | TargetAvg=142.8 |
MAE=17.08
[Warmstart 13/15] Step 160/300 | Loss=0.2663 | PredAvg=137.2 | TargetAvg=143.2 |
MAE=17.29
[Warmstart 13/15] Step 180/300 | Loss=0.2855 | PredAvg=137.6 | TargetAvg=144.2 |
MAE=18.27
[Warmstart 13/15] Step 200/300 | Loss=0.2602 | PredAvg=137.6 | TargetAvg=143.3 |
MAE=17.36
[Warmstart 13/15] Step 220/300 | Loss=0.2490 | PredAvg=138.1 | TargetAvg=143.9 |
MAE=17.41
[Warmstart 13/15] Step 240/300 | Loss=0.2601 | PredAvg=138.6 | TargetAvg=144.0 |
MAE=17.99
[Warmstart 13/15] Step 260/300 | Loss=0.2682 | PredAvg=138.4 | TargetAvg=144.0 |
MAE=18.05
[Warmstart 13/15] Step 280/300 | Loss=0.2552 | PredAvg=138.2 | TargetAvg=143.6 |
MAE=17.63
[Warmstart 13/15] Step 300/300 | Loss=0.2711 | PredAvg=138.2 | TargetAvg=144.2 |
MAE=18.09
Warmstart epoch 13 done | Loss=0.2711
  Det eval complete | Acc=0.2100 | AvgBudget=143.4 | StdBudget=7.1 | Range=[134,
```

```
166]
[Warmstart 14/15] Step 20/300 | Loss=0.3122 | PredAvg=141.1 | TargetAvg=145.4 |
MAE=21.25
[Warmstart 14/15] Step 40/300 | Loss=0.3766 | PredAvg=135.8 | TargetAvg=142.7 |
MAE=17.55
[Warmstart 14/15] Step 60/300 | Loss=0.2787 | PredAvg=134.6 | TargetAvg=140.0 |
MAE=15.03
[Warmstart 14/15] Step 80/300 | Loss=0.2579 | PredAvg=133.8 | TargetAvg=140.3 |
MAE=14.55
[Warmstart 14/15] Step 100/300 | Loss=0.2697 | PredAvg=135.5 | TargetAvg=141.0 |
MAE=15.63
[Warmstart 14/15] Step 120/300 | Loss=0.2751 | PredAvg=135.2 | TargetAvg=141.7 |
MAE=16.09
[Warmstart 14/15] Step 140/300 | Loss=0.3046 | PredAvg=136.1 | TargetAvg=143.0 |
MAE=17.39
[Warmstart 14/15] Step 160/300 | Loss=0.2740 | PredAvg=135.9 | TargetAvg=141.9 |
MAE=16.34
[Warmstart 14/15] Step 180/300 | Loss=0.2835 | PredAvg=135.8 | TargetAvg=142.6 |
MAE=16.99
[Warmstart 14/15] Step 200/300 | Loss=0.2654 | PredAvg=136.3 | TargetAvg=142.7 |
MAE=16.82
[Warmstart 14/15] Step 220/300 | Loss=0.2901 | PredAvg=136.8 | TargetAvg=143.7 |
MAE=17.52
[Warmstart 14/15] Step 240/300 | Loss=0.2946 | PredAvg=136.8 | TargetAvg=144.2 |
MAE=17.61
[Warmstart 14/15] Step 260/300 | Loss=0.2801 | PredAvg=137.3 | TargetAvg=144.2 |
MAE=17.47
[Warmstart 14/15] Step 280/300 | Loss=0.2788 | PredAvg=137.4 | TargetAvg=144.5 |
MAE=17.70
[Warmstart 14/15] Step 300/300 | Loss=0.2731 | PredAvg=137.2 | TargetAvg=144.2 |
MAE=17.49
Warmstart epoch 14 done | Loss=0.2731
  Det eval complete | Acc=0.2400 | AvgBudget=133.8 | StdBudget=4.7 | Range=[129,
156]
[Warmstart 15/15] Step 20/300 | Loss=0.3689 | PredAvg=135.8 | TargetAvg=147.0 |
MAE=18.95
[Warmstart 15/15] Step 40/300 | Loss=0.3118 | PredAvg=136.6 | TargetAvg=144.9 |
MAE=18.62
[Warmstart 15/15] Step 60/300 | Loss=0.2981 | PredAvg=135.1 | TargetAvg=143.6 |
MAE=17.30
[Warmstart 15/15] Step 80/300 | Loss=0.2874 | PredAvg=135.1 | TargetAvg=142.4 |
MAE=16.66
[Warmstart 15/15] Step 100/300 | Loss=0.2440 | PredAvg=135.0 | TargetAvg=141.2 |
MAE=15.57
[Warmstart 15/15] Step 120/300 | Loss=0.2581 | PredAvg=136.3 | TargetAvg=143.3 |
MAE=16.45
[Warmstart 15/15] Step 140/300 | Loss=0.2497 | PredAvg=138.9 | TargetAvg=144.2 |
MAE=17.54
```

```
[Warmstart 15/15] Step 160/300 | Loss=0.2519 | PredAvg=138.6 | TargetAvg=144.2 |
MAE=17.66
[Warmstart 15/15] Step 180/300 | Loss=0.2761 | PredAvg=138.9 | TargetAvg=145.5 |
MAE=18.07
[Warmstart 15/15] Step 200/300 | Loss=0.2579 | PredAvg=139.9 | TargetAvg=144.8 |
MAE=18.08
[Warmstart 15/15] Step 220/300 | Loss=0.2538 | PredAvg=139.0 | TargetAvg=144.6 |
MAE=17.68
[Warmstart 15/15] Step 240/300 | Loss=0.2648 | PredAvg=139.0 | TargetAvg=144.6 |
MAE=17.87
[Warmstart 15/15] Step 260/300 | Loss=0.2720 | PredAvg=139.0 | TargetAvg=144.8 |
MAE=17.92
[Warmstart 15/15] Step 280/300 | Loss=0.2673 | PredAvg=138.7 | TargetAvg=144.7 |
MAE=17.78
[Warmstart 15/15] Step 300/300 | Loss=0.2597 | PredAvg=138.5 | TargetAvg=144.2 |
MAE=17.48
Warmstart epoch 15 done | Loss=0.2597
  Det eval complete | Acc=0.2400 | AvgBudget=134.7 | StdBudget=5.5 | Range=[129,
159]

Loading best warm-start policy by accuracy…

Post warm-start evaluation:
  Det eval complete | Acc=0.2900 | AvgBudget=147.8 | StdBudget=10.6 |
Range=[129, 171]
{
  "warmstart_eval": {
    "accuracy": 0.29,
    "avg_thinking_tokens": 147.84,
    "avg_reward": 0.28858,
    "avg_budget": 147.84,
    "min_budget": 129.0,
    "max_budget": 171.0,
    "std_budget": 10.636465578377058
  }
}

Pre-computing all training features…

Pre-training value network on oracle rewards…
  Target oracle avg reward: 0.6446
  Value pretrain epoch 1/5 | Loss=2.6755
  Value pretrain epoch 2/5 | Loss=0.8375
  Value pretrain epoch 3/5 | Loss=0.6598
  Value pretrain epoch 4/5 | Loss=0.6835
  Value pretrain epoch 5/5 | Loss=0.4889
Value pre-training complete. PPO will start with better advantage estimates.
```

```
Starting PPO training…
  PPO epsilon (clip range): 0.2
  PPO epochs per rollout: 2
  Batch size: 16 questions at once on GPU
  Pre-encoding all training features…

================================================================================
PPO Epoch 1/10
================================================================================
  Phase 1: Collecting rollout experiences (batched)…
  Rollout done | Acc=0.3133 | AvgReward=0.3115 | AvgBudget=160.3 |
StdBudget=26.4
  Phase 2: PPO updates (2 epochs, batch=32)…
    PPO update epoch 1/2 | PolicyLoss=0.0146 | ValueLoss=1.4261 | ClipFrac=0.056
    PPO update epoch 2/2 | PolicyLoss=0.0002 | ValueLoss=0.5755 | ClipFrac=0.167

  Evaluating deterministic policy on test set…
  Det eval complete | Acc=0.3000 | AvgBudget=150.6 | StdBudget=11.0 |
Range=[130, 173]
  Test Accuracy  : 0.3000
  Test AvgTokens : 150.6
  Test AvgReward : 0.2997
  Test Budget    : 150.6 +/- 11.0 [130, 173]
  Saved best PPO model by reward = 0.2997
  Saved best PPO model by accuracy = 0.3000

================================================================================
PPO Epoch 2/10
================================================================================
  Phase 1: Collecting rollout experiences (batched)…
  Rollout done | Acc=0.3033 | AvgReward=0.2986 | AvgBudget=161.1 |
StdBudget=26.1
  Phase 2: PPO updates (2 epochs, batch=32)…
    PPO update epoch 1/2 | PolicyLoss=0.0319 | ValueLoss=0.4224 | ClipFrac=0.028
    PPO update epoch 2/2 | PolicyLoss=-0.0072 | ValueLoss=0.6279 |
ClipFrac=0.134

  Evaluating deterministic policy on test set…
  Det eval complete | Acc=0.2900 | AvgBudget=154.7 | StdBudget=11.0 |
Range=[131, 176]
  Test Accuracy  : 0.2900
  Test AvgTokens : 154.7
  Test AvgReward : 0.2851
  Test Budget    : 154.7 +/- 11.0 [131, 176]

================================================================================
PPO Epoch 3/10
```

```
================================================================================
  Phase 1: Collecting rollout experiences (batched)…
  Rollout done | Acc=0.3200 | AvgReward=0.3196 | AvgBudget=160.9 |
StdBudget=24.8
  Phase 2: PPO updates (2 epochs, batch=32)…
    PPO update epoch 1/2 | PolicyLoss=0.0057 | ValueLoss=0.6579 | ClipFrac=0.000
    PPO update epoch 2/2 | PolicyLoss=0.0019 | ValueLoss=0.3353 | ClipFrac=0.139

  Evaluating deterministic policy on test set…
  Det eval complete | Acc=0.2900 | AvgBudget=160.3 | StdBudget=10.6 |
Range=[135, 179]
  Test Accuracy  : 0.2900
  Test AvgTokens : 160.3
  Test AvgReward : 0.2823
  Test Budget    : 160.3 +/- 10.6 [135, 179]

================================================================================
PPO Epoch 4/10
================================================================================
  Phase 1: Collecting rollout experiences (batched)…
  Rollout done | Acc=0.3067 | AvgReward=0.3013 | AvgBudget=164.0 |
StdBudget=25.7
  Phase 2: PPO updates (2 epochs, batch=32)…
    PPO update epoch 1/2 | PolicyLoss=0.0199 | ValueLoss=0.3195 | ClipFrac=0.033
    PPO update epoch 2/2 | PolicyLoss=0.0248 | ValueLoss=0.3242 | ClipFrac=0.009

  Evaluating deterministic policy on test set…
  Det eval complete | Acc=0.3000 | AvgBudget=161.8 | StdBudget=10.3 |
Range=[136, 179]
  Test Accuracy  : 0.3000
  Test AvgTokens : 161.8
  Test AvgReward : 0.2941
  Test Budget    : 161.8 +/- 10.3 [136, 179]

================================================================================
PPO Epoch 5/10
================================================================================
  Phase 1: Collecting rollout experiences (batched)…
  Rollout done | Acc=0.3100 | AvgReward=0.3053 | AvgBudget=164.3 |
StdBudget=25.4
  Phase 2: PPO updates (2 epochs, batch=32)…
    PPO update epoch 1/2 | PolicyLoss=-0.0195 | ValueLoss=0.3057 |
ClipFrac=0.054
    PPO update epoch 2/2 | PolicyLoss=-0.0611 | ValueLoss=0.2915 |
ClipFrac=0.237

  Evaluating deterministic policy on test set…
  Det eval complete | Acc=0.3500 | AvgBudget=165.6 | StdBudget=9.5 | Range=[140,
```

```
181]
  Test Accuracy  : 0.3500
  Test AvgTokens : 165.6
  Test AvgReward : 0.3547
  Test Budget    : 165.6 +/- 9.5 [140, 181]
  Saved best PPO model by reward = 0.3547
  Saved best PPO model by accuracy = 0.3500

================================================================================
PPO Epoch 6/10
================================================================================
  Phase 1: Collecting rollout experiences (batched)…
  Rollout done | Acc=0.3200 | AvgReward=0.3171 | AvgBudget=165.9 |
StdBudget=27.0
  Phase 2: PPO updates (2 epochs, batch=32)…
    PPO update epoch 1/2 | PolicyLoss=-0.0124 | ValueLoss=0.3024 |
ClipFrac=0.003
    PPO update epoch 2/2 | PolicyLoss=-0.0263 | ValueLoss=0.4322 |
ClipFrac=0.056

  Evaluating deterministic policy on test set…
  Det eval complete | Acc=0.3000 | AvgBudget=167.8 | StdBudget=9.0 | Range=[142,
182]
  Test Accuracy  : 0.3000
  Test AvgTokens : 167.8
  Test AvgReward : 0.2911
  Test Budget    : 167.8 +/- 9.0 [142, 182]

================================================================================
PPO Epoch 7/10
================================================================================
  Phase 1: Collecting rollout experiences (batched)…
  Rollout done | Acc=0.3367 | AvgReward=0.3373 | AvgBudget=167.0 |
StdBudget=26.2
  Phase 2: PPO updates (2 epochs, batch=32)…
    PPO update epoch 1/2 | PolicyLoss=-0.0129 | ValueLoss=0.3653 |
ClipFrac=0.009
    PPO update epoch 2/2 | PolicyLoss=0.0054 | ValueLoss=0.3402 | ClipFrac=0.035

  Evaluating deterministic policy on test set…
  Det eval complete | Acc=0.3100 | AvgBudget=174.2 | StdBudget=6.9 | Range=[153,
184]
  Test Accuracy  : 0.3100
  Test AvgTokens : 174.2
  Test AvgReward : 0.3004
  Test Budget    : 174.2 +/- 6.9 [153, 184]

================================================================================
```

```
PPO Epoch 8/10
================================================================================
  Phase 1: Collecting rollout experiences (batched)…
  Rollout done | Acc=0.3567 | AvgReward=0.3594 | AvgBudget=172.8 |
StdBudget=27.5
  Phase 2: PPO updates (2 epochs, batch=32)…
    PPO update epoch 1/2 | PolicyLoss=-0.0306 | ValueLoss=0.3290 |
ClipFrac=0.099
    PPO update epoch 2/2 | PolicyLoss=-0.0566 | ValueLoss=0.3508 |
ClipFrac=0.128

  Evaluating deterministic policy on test set…
  Det eval complete | Acc=0.3300 | AvgBudget=174.7 | StdBudget=6.7 | Range=[154,
185]
  Test Accuracy  : 0.3300
  Test AvgTokens : 174.7
  Test AvgReward : 0.3251
  Test Budget    : 174.7 +/- 6.7 [154, 185]

================================================================================
PPO Epoch 9/10
================================================================================
  Phase 1: Collecting rollout experiences (batched)…
  Rollout done | Acc=0.3600 | AvgReward=0.3630 | AvgBudget=174.0 |
StdBudget=25.8
  Phase 2: PPO updates (2 epochs, batch=32)…
    PPO update epoch 1/2 | PolicyLoss=0.0071 | ValueLoss=0.2985 | ClipFrac=0.033
    PPO update epoch 2/2 | PolicyLoss=-0.0151 | ValueLoss=0.3182 |
ClipFrac=0.013

  Evaluating deterministic policy on test set…
  Det eval complete | Acc=0.3700 | AvgBudget=178.7 | StdBudget=5.2 | Range=[162,
186]
  Test Accuracy  : 0.3700
  Test AvgTokens : 178.7
  Test AvgReward : 0.3732
  Test Budget    : 178.7 +/- 5.2 [162, 186]
  Saved best PPO model by reward = 0.3732
  Saved best PPO model by accuracy = 0.3700

================================================================================
PPO Epoch 10/10
================================================================================
  Phase 1: Collecting rollout experiences (batched)…
  Rollout done | Acc=0.3567 | AvgReward=0.3575 | AvgBudget=176.6 |
StdBudget=26.1
  Phase 2: PPO updates (2 epochs, batch=32)…
    PPO update epoch 1/2 | PolicyLoss=-0.0271 | ValueLoss=0.2800 |
```

```
ClipFrac=0.000
    PPO update epoch 2/2 | PolicyLoss=-0.0181 | ValueLoss=0.3419 |
ClipFrac=0.040

  Evaluating deterministic policy on test set…
  Det eval complete | Acc=0.3900 | AvgBudget=182.0 | StdBudget=3.8 | Range=[170,
187]
  Test Accuracy  : 0.3900
  Test AvgTokens : 182.0
  Test AvgReward : 0.3965
  Test Budget    : 182.0 +/- 3.8 [170, 187]
  Saved best PPO model by reward = 0.3965
  Saved best PPO model by accuracy = 0.3900

Loading best PPO model for final evaluation…

Final deterministic policy evaluation:
  Det eval complete | Acc=0.3900 | AvgBudget=182.0 | StdBudget=3.8 | Range=[170,
187]
{
  "final_deterministic_eval": {
    "accuracy": 0.39,
    "avg_thinking_tokens": 181.96,
    "avg_reward": 0.39651999999999993,
    "avg_budget": 181.96,
    "min_budget": 170.0,
    "max_budget": 187.0,
    "std_budget": 3.7680764323458193
  }
}

Saved: final_policy_v13.pt
Saved: final_value_v13.pt

========================================================================
FINAL RESULTS COMPARISON
========================================================================
Method                                 Acc  AvgTokens  AvgReward
------------------------------------------------------------------------
  Fixed budget 128                    0.2500       128     0.2485
  Fixed budget 144                    0.2400       144     0.2280
  Fixed budget 160                    0.2500       160     0.2325
  Fixed budget 168                    0.2900       168     0.2785
  Fixed budget 176                    0.3500       176     0.3495
  Fixed budget 184                    0.3900       184     0.3955
  Fixed budget 192                    0.4100       192     0.4165
  Fixed budget 210                    0.4500       210     0.4575
  Fixed budget 220                    0.4200       220     0.4150
```

```
  Policy (PPO deterministic)     0.3900        182     0.3965
============================================================================

AI Disclosure: Written with Claude's assistance
```

[ ]: